9

Creating Single- and Multi-Agent Systems

In previous chapters, we discussed a number of components or tools that can be associated with LLMs to extend their capabilities. In *Chapters 5* and *6*, we addressed in detail how external memory can be used to enrich the context. This allows the model to obtain additional information to be able to answer user questions when it does not know the answer (when it hasn't seen the document during pre-training or it relates to information after the date of their training). Similarly, in ***Chapter 7***, we saw that knowledge graphs can be used to extend the model's knowledge. These components attempt to solve one of the most problematic limitations of LLMs, namely, hallucinations (an output produced by the model that is not factually correct). In addition, we saw that the use of graphs allows the model to conduct graph reasoning and thus adds new capabilities.

In ***Chapter 8***, we saw the intersection of RL and LLMs. One of the problems associated with LLMs is that they could produce harmful content (such as biased or toxic content or misinformation). RL algorithms allow us to align the behavior of the model with human preferences, thus allowing us to reduce the risk of harmful content.

We can use similar approaches to make the model more capable of performing tasks or following instructions. In the future, these reinforcement learning algorithms could be useful in overcoming an important limitation of LLMs: a lack of continual learning.

The definition of tools, as we will see, is quite broad. In fact, any software or algorithm can be a tool. As we have already seen in previous chapters, LLMs can execute code or connect to **application programming Interfaces** (**APIs**). But this means that they can also invoke other models to perform tasks that they are unable to accomplish on their own.

In any case, all these elements have set the seed for what is called the agent revolution, in which an LLM can interact with the environment and perform tasks in the real world (be it the internet or, in the future, beyond the constraint of a computer).

In this chapter, we focus on LLMs, its various tools, and how these can be combined to interact with the environment. We will start with the definition of an autonomous agent and continue with what the tools (APIs, models, and so on) are and how they can be organized. We will see how using prompt engineering techniques (which we addressed in ***Chapter 3***) allows us to create different types of agents. After that, we will discuss several strategies that have been used previously in the literature to connect an LLM to its tools.

This will allow us to see in detail how some technical limitations and challenges have been solved. We will then talk in detail about HuggingGPT (an LLM connected to hundreds of models), which was a turning point in agent creation. We will see how HuggingGPT allows an LLM to solve complex tasks using other expert models. Then, we will see how instead of a single agent, we can create multi-agent platforms. The interaction of different agents will allow us to solve increasingly complex tasks and issues. In addition, we will see how these approaches can be applied to complex domains, such as healthcare, chemistry, and law. We will then put what we have seen into practice using HuggingGPT. Next, we will extend this concept with a multi-agent platform that will allow us to understand how modern systems work.

Once we have seen how agents or multi-agents work, we will discuss in detail the new business paradigms that are emerging, such as **Software as a Service** (**SaaS**), **Model as a Service** (**MaaS**), **Data as a Service** (**DaaS**), and **Results as a Service** (**RaaS**) or **Outcome as a Service** (**OaaS**). As we will see in this chapter, each of these business models has advantages and disadvantages.

In this chapter, we'll be covering the following topics:

- Introduction to autonomous agents
- HuggingGPT and other approaches
- Working with HuggingGPT
- Multi-agent system
- SaaS, MaaS, DaaS, and RaaS

Technical requirements

The code in this chapter requires the use of a GPU. For the section on using HuggingGPT in particular, both a GPU and plenty of space on the hard disk drive are required (several models will be downloaded, including diffusion models. For this, it will be necessary to use Git **Large File Storage** (**LFS**), which allows downloading wide files via Git). Anaconda should be installed to obtain the various libraries (the necessary libraries will be set up directly during installation). For readers who do not have these resources, the *Using HuggingGPT on the web* section shows how you can use HuggingGPT on the web. For local use of HuggingGPT, it is necessary to have an OpenAI token, while for web use, it is also necessary to have a Hugging Face token. The multi-agent system is based on Python libraries (NumPy, scikit-learn, SentenceTransformers, and Transformers).

HuggingGPT should be run on a GPU. The multi-agent system should be run on a GPU, but it could also be run on a CPU; this is, however, highly discouraged. The code can be found on GitHub: **https://github.com/PacktPublishing/Modern-AI-Agents/tree/main/chr9**.

Introduction to autonomous agents

In the context of AI, **autonomous agents** refer to systems or entities that can perform tasks or make decisions independently without the need for human intervention. These agents are designed to perceive their environment, reason about it, make decisions based on their goals, and take action accordingly to achieve those goals. Autonomous agents are consid-

ered an important step toward **artificial general intelligence** (**AGI**), which is expected to conduct autonomous planning and actions.

The main reason for using LLMs as agents lies in the fact that LLMs have shown some reasoning and thus planning capabilities. LLMs use reasoning to interpret input, draw inferences, and make decisions (showing some extent of deductive, inductive, and abductive reasoning). This allows LLMs to apply general rules to specific cases (deductive reasoning), learn patterns from examples (inductive reasoning), and infer explanations from incomplete data (abductive reasoning). In addition, LLMs are capable of conducting step reasoning by chaining ideas, thus enabling them to be able to solve equations or debug code. Also, solving some problems (such as math problems) requires following a series of steps. Intrinsically, an LLM must often decompose a task into a series of actions, anticipate the results of these actions, and adjust its behavior in response to the results. These capabilities, however, are limited to the context provided by the user or knowledge gained during pre-training, and for fields such as medicine or finance, this is not enough to solve most problems. Therefore, the natural response to this limitation is to extend the capabilities of the LLM with external tools, or otherwise connect an LLM to the external environment.

The purpose of some studies and research is therefore to extend the capabilities of LLMs with a set of tools. These works and derived libraries try to equip LLMs with human capabilities, such as memory and planning, to make them behave like humans and complete various tasks effectively.

As the capabilities of LLMs have developed, interest in these agents has grown, and numerous articles and frameworks have been published.

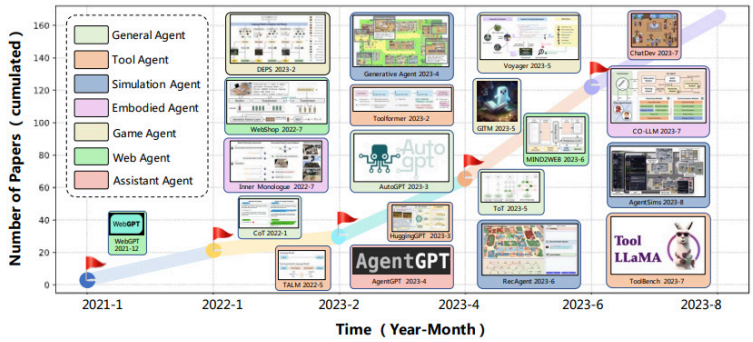


Figure 9.1 – Growing interest in LLM autonomous agents (https://arxiv.org/pdf/2308.11432)

The first aspect to consider when building these types of systems is the design of the architecture and how to use it to perform tasks. Autonomous agents must perform different roles, perceive the environment, and learn from it. The purpose of the architecture is to assist an LLM in maximizing its capabilities in order to be used as an agent. To this end, several modules have been developed, which can be divided into four main groups: profiling, memory, planning, and action.

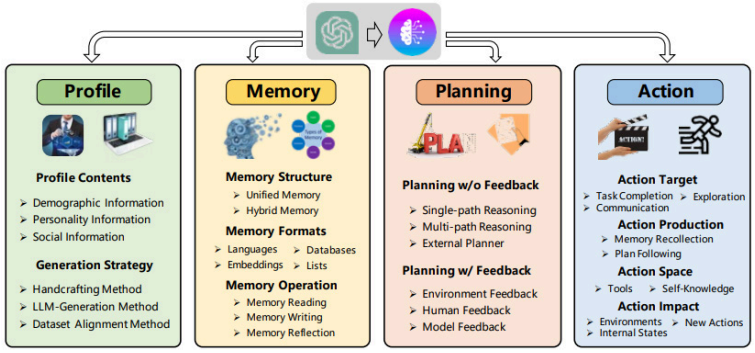


Figure 9.2 – Possible modules to build LLM-based autonomous agents (https://arxiv.org/pdf/2308.11432)

Let's go through each of these in a bit more detail:

- **Profiling module**: Often, agents perform tasks in specific roles (also called personas), such as coders, domain experts, teachers, or assistants. The profiling module deals with defining these roles (characteristics, role, psychological and social information, and relationships with other agents) in a specific prompt given to the LLM. These profiles can then be handwritten (handwritten profiles are manually crafted personas or roles defined by developers or domain experts); for example, for a system for software development, we can create different job roles ("you are a software engineer responsible for code review"). Handwritten profiles allow a high degree of control, enriching context, and can be highly domain-specific (addressing nuances, soft skills, sophisticated knowledge). Although the handwritten approach is very flexible, it is time-consuming and has limited scalability. So, some studies have explored systems where LLMs automatically generate profiles (using few-shot examples, rules, and templates, or specific external datasets as job descriptions). This approach is much more scalable and adaptable to different situations (especially if the system is to be dynamic or if feedback is received from users). On the other hand, however, there is less control (the system loses nuance and depth, with the risk of being generic), the quality is variable (depending on the prompt engineering technique, some examples might be of poor quality), and it still requires verification by a human.
- **Memory module**: The memory module stores information perceived by the system from the environment or other sources; these memories then facilitate future actions. Dedicated memory components can also be sophisticated and inspired by human cognition, with components dedicated to perceptual, short- or long-term information. Commonly found memories are then entered into the system prompt (so the context length of the LLM is the limit for the memory that can be used for the agent). An example is the history of chats with a user that is needed for task accomplishment. As another example, an agent assisting in the development of a game will have just-occurring events and other descriptions as short-term memory. **Hybrid memory** is a way of extending memory, where past events and thoughts are saved and found again to facilitate the agent's behavior. Hybrid memory combines short-term (within an LLM context) and long-term (external) memory to extend the agent's capacity beyond the LLM's context window. These thoughts, conversations, or other information can be saved via RAG or other systems (database, knowledge graph, and so on). When needed, relevant information is retrieved and injected into the LLM prompt, allowing the agent to act on prior knowledge without exceed-

ing context limits. For example, in RAG, a search mechanism pulls relevant documents or memory fragments based on the current query, making responses more informed and consistent over time. In addition, this module should cover three operations: memory reading (extracting useful information for the agent's action), memory writing (storing information about the environment that may be useful in the future while avoiding duplicates and memory overflow), and memory reflection (evaluating and inferring more abstract, complex, and high-level information). Specifically, memory reading retrieves information to support the agent's decisions (increasing context continuity and consistency), memory writing allows for saving information that is useful for the agent's interaction with the environment (thus reducing redundancy and allowing for overcoming the limitations of a noneditable memory), and memory reflection allows for deriving insights from the analysis of stored information, thus allowing for adjusting behavior to achieve goals.

- **Planning module**: The planning module is generally used to deconstruct complex tasks into more manageable tasks, to make LLMs behave more reasonably, powerfully, and reliably. The planning module can include or not include feedback.
  In planning without feedback, the agent does not receive feedback that influences its future behavior after it has conducted an action. In single-path reasoning, the task is divided into several intermediate steps connected in a cascading sequence. **Chain of thought** (**CoT**) reasoning is often employed to develop a step-by-step plan for this strategy. In contrast, multi-path reasoning involves a tree-like structure where each intermediate step can branch into multiple subsequent steps. These approaches typically leverage **self-consistent CoT** (**CoT-SC**) or **tree of thoughts** (**ToT**) frameworks, enabling the evaluation of all intermediate steps to identify the optimal strategy. The tree can be even coupled with sophisticated strategies such as **Monte Carlo Tree Search** (**MCTS**) or an external planner.
  Planning with feedback is mainly used for long-term tasks, where it is difficult to generate an effective plan from the beginning or the dynamics may change. So, you can incorporate feedback from the environment and observations. For example, the ReAct framework uses thought-act-observation triplets. Another alternative is using human feedback or another model to improve the agent's planning ability.

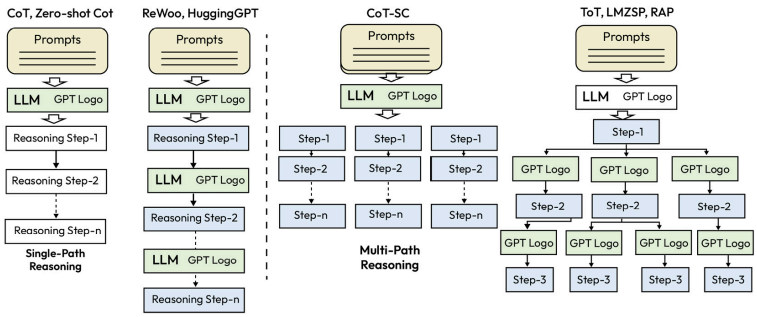


Figure 9.3 – Comparison between the strategies of single-path and multi-path reasoning (https://arxiv.org/pdf/2308.11432)

- **Action module**: The action module is responsible for translating the planning into a specific outcome; this module is then responsible for the interaction. In general, this module focuses on the execution of the task and then actions with a specific goal. The module is also responsible for communicating with other agents (if they are present), explor-

ing the environment, finding the necessary memory, and executing the plan. To accomplish these goals, the LLM can use either the knowledge gained from the LLM during the pre-training phase or external tools (external models, APIs, databases, or other tools). Pre-training knowledge allows the LLM to carry out many tasks using learned information, such as generating text, answering questions, or making decisions based on prior data. However, for more dynamic, real-time, or specialized tasks, the action module uses external tools such as APIs, databases, software applications, or other models. These tools enable the agent to access up-to-date information, manipulate data, perform calculations, or trigger operations in external systems. Together, pre-trained knowledge and external tools allow the agent to interact meaningfully with its environment, carry out goals, and adapt based on the outcomes of its actions. The action of the model has an impact on the environment or internal state of the model, and this is evaluated and taken into account by this module.

Apart from system architecture, we should also consider strategies to develop better agents. Typically, one of the most used strategies is conducting fine-tuning of the model. Fine-tuning plays a key role in improving agent performance by adapting a general-purpose LLM to specific tasks, domains, or behavioral goals. It helps align the model with human values (safety), improve instruction following, or specialize in areas such as education or e-commerce. In most cases, human-annotated datasets are used for specific tasks. As we discussed in ***Chapter 3***, this can be for security reasons (alignment with human values), to make it more responsive to following instructions (instruction tuning), or to train to a specific domain or task. To fine-tune an agent, in the WebShop example (**https://arxiv.org/pdf/2308.11432**), the authors of the paper collected 1.2 million world products from **amazon.com** and created a simulated e-commerce website. After that, they collected human behaviors on the website (when users browse and perform actions on the website, their behaviors are registered), thus creating a dataset for fine-tuning specifically for an agent dedicated to helping with product selection. Or, in the EduChat example (**https://arxiv.org/pdf/2308.11432**), to create an agent for educational scenarios, the authors collected an annotated dataset covering various educational scenarios (the dataset was evaluated and edited by specialized personnel, such as psychologists).

Collecting these datasets is expensive and requires specialized personnel in several cases. Therefore, an alternative is to use an LLM to annotate the dataset. When this approach is followed, there is a trade-off between quality and cost: the dataset is not as good as that annotated by humans, but the costs are much reduced. For example, in ToolBench (an agent system where the LLM is connected to APIs), the authors of that work (**https://arxiv.org/pdf/2308.11432**) collected more than 16,000 real-world APIs and then annotated this dataset with ChatGPT. Then, they fine-tuned LLaMA on this dataset. The fine-tuned model was much more performant in using these APIs.

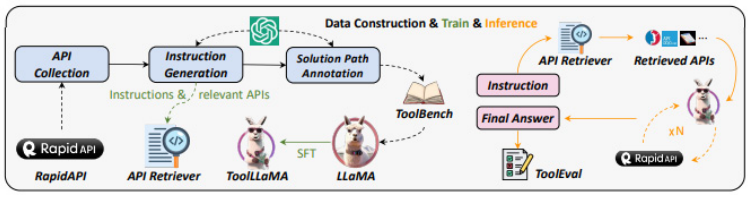


Figure 9.4 – Construction of ToolBench (**https://arxiv.org/pdf/2307.16789**)

Alternatively, you can collect a large amount of data that is not annotated, so that the model figures out on its own during fine-tuning . For example, Mind2Web collected a large amount of data for web browsing (https://arxiv.org/abs/2306.06070).

The trade-off between annotated and self-annotated datasets is that LLM-labeled data may lack the accuracy, nuance, or reliability of human annotation, potentially affecting performance. Still, it allows broader coverage and faster iteration. In practice, combining both methods—using LLMs for bulk labeling and humans for validation or high-stakes tasks—offers a balance between quality and cost, making fine-tuning more accessible while still enhancing agent capabilities.

Because interactions with the model are typically conducted with the prompt, many developers simply use prompt engineering without the need for fine-tuning. The rationale is that the necessary knowledge already exists in the parameters of the LLM and we want to use a prompt that allows the model to use it to its best advantage. Other approaches add agents that act as critics, other agents that debate, or other variations.

What we have seen so far enables us to understand what an autonomous agent is and how it is composed. As we have seen, an agent has, at its core, an LLM and a sophisticated ecosystem around it that can be composed of different elements as the researcher chooses. In the following sections, we will look in detail at different approaches to autonomous agents that allow us to understand some of the solutions that have been implemented in the literature.

Toolformer

Toolformer (Schick, 2023) is a pioneering work using the idea that an LLM can access external tools to solve tasks (search engines, calculators, and calendars) without sacrificing their generality or requiring large-scale human annotation. The key innovation of Toolformer lies in treating tool use as a generalizable skill, not bound to a specific task. Rather than designing separate systems for each tool or task, Toolformer teaches the model to make intelligent decisions about which tool to use, when to use it, and how to use it, all within a unified language modeling framework.

According to the authors, an LLM should learn the use of tools according to two principles: in a self-supervised way and preserving the generality of the model. Toolformer is designed to learn in a largely self-supervised manner, addressing a major bottleneck in AI development: the cost and effort of human-labeled data. Instead of manually annotating data with tool usage, the model is shown a few examples of how tools (API calls) work. It then automatically annotates a large, unlabeled dataset with tool-use opportunities during language modeling. These annotated sequences are used to fine-tune the model, enabling it to learn tool interactions naturally. This is important because there is a cost associated with annotating a dataset, but it also teaches an LLM how to use the tools. A central goal is to ensure that the LLM retains its broad capabilities across tasks while gaining the ability to use tools. Tool use is not hardcoded for specific prompts—it becomes part of the model's general skillset. The LLM learns when a tool improves performance and chooses to invoke it only when necessary, maintaining flexibility and avoiding over-dependence.

In short, tool use is not associated with a specific task but becomes a general concept. The idea behind Toolformer is it is a model that treats a tool as a call to an API. This abstraction simplifies integration and scales easily to different tools. For instance, the model might decide to call a calculator API when faced with a math problem or a search engine when external knowledge is needed. Given a series of human-written examples of how an API can be used, the authors used an LLM to annotate a huge language modeling dataset with potential API calls. After that, the authors conduct fine-tuning of the model to improve the model's capabilities. With this approach, an LLM learns how to control a variety of tools and when it should use them.



Figure 9.5 – Toolformer approach (https://arxiv.org/pdf/2302.04761)

HuggingGPT

HuggingGPT (Shen, 2023) introduces a powerful concept: using language as a generic interface that enables LLMs to collaborate with external AI models across various modalities, such as vision, speech, and structured data. Instead of being limited to textual tasks, the LLM gains the ability to manage and orchestrate other models to solve complex, real-world problems. HuggingGPT is based on two ideas: an LLM is limited if it cannot access information beyond text (such as vision and speech), and in the real world, complex tasks can be decomposed into smaller tasks that are more manageable. For specific tasks, LLMs have excellent capabilities in zero-shot or few-shot learning, but generalist models are less capable than specific trained models. So, for the authors, the solution is that an LLM must be able to coordinate with external models to harness their powers. In the article, they focus on finding suitable middleware to bridge the connections between LLMs and AI models. In other words, the idea is that LLMs can dialogue with other models and thus exploit their capabilities. The intuition behind it is that each AI model can be described in the form of language by summarizing its function. In other words, each model can be described functionally and textually. This description can then be used by an LLM. For the authors, this represents the introduction of a new concept: *Language as a generic interface for LLMs to collaborate with AI models*. In this system, the LLM acts as the "brain," responsible for interpreting the user's request, decomposing it into subtasks, selecting the appropriate models based on their textual descriptions, scheduling and coordinating model execution, integrating results, and generating a final response.

Since interaction with an LLM is through a prompt, a model's function description can be entered in the LLM prompt. An LLM then can be seen as the brain that manages AI models for planning, scheduling, and cooperation. So, an LLM does not accomplish the task directly but invokes specific models to solve tasks. For example, if a user asks, "*What animal is in the image?*", the LLM processes the question and reasons what type of model it should use (i.e., an image classifier); the model is invoked, which returns an output (the animal present), and the LLM generates a textual output to answer "*the animal is a chicken*."

At this point, the main problem is collecting these textual descriptions of the functions of the models. Fortunately, the **machine learning** (**ML**) community provides quality descriptions for specific tasks and the models used to solve them (language, vision, speech, and so on). So, what we need is to tie LLMs to the community (GitHub, Hugging Face, and so on).

In short, HuggingGPT is an LLM-powered agent designed to solve a variety of complex tasks autonomously. HuggingGPT connects an LLM (in the original article, it is ChatGPT) with the ML community (Hugging Face, but the principle can be generalized); the LLM can take different modalities as input and accomplish different tasks. The LLM acts as a brain, divides the user's request into subtasks, and then assigns them to specialized models (in accordance with the model description); it then executes these models and integrates the results. These principles are highlighted in the following figure:

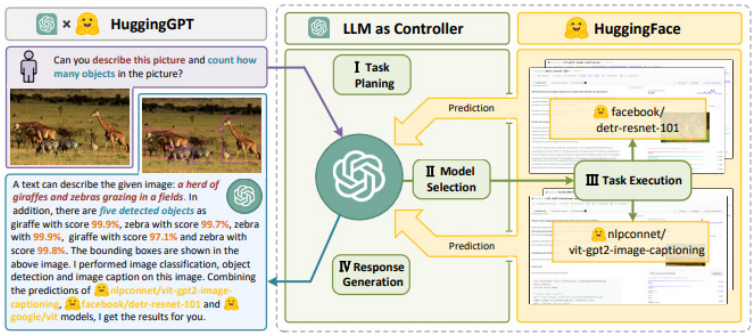


Figure 9.6 – HuggingGPT general scheme (https://arxiv.org/pdf/2303.17580)

The whole HuggingGPT process can then be divided into four steps:

1. Task planning: ChatGPT analyzes the requests by the user (understands the intention) and transforms the question into possible solvable tasks.
2. Model selection: ChatGPT selects the appropriate models (expert models) that are present in Hugging Face (the models are selected based on the provided description).
3. Task execution: The model is invoked and executed, and then the results are returned to ChatGPT.
4. Response generation: ChatGPT integrates the results of the models and generates the answers.

In Toolformer, we have an LLM where the model calls a tool via an API call. HuggingGPT uses a similar approach but without the need for fine-tuning. In HuggingGPT, an LLM can be seen as a controller that routes user requests to expert models. In other words, the LLM understands the task and plans the action, but this action is then conducted by expert models (the LLM just integrates the results). The LLM here is just a facilitator that organizes the cooperation of different models to solve different tasks in different domains. The LLM then maintains its generality and can choose which tool to use and when to use it (in this case, the models are the tools). For example, if an LLM does not have capabilities in a certain mode, it exploits the capabilities of an expert model to be able to accomplish the task. The LLM just needs to know which model to call to solve a specific task. HuggingGPT thus represents a flexible system, where we only need textual descriptions to provide to the LLM, and then the LLM will integrate the different expert models.

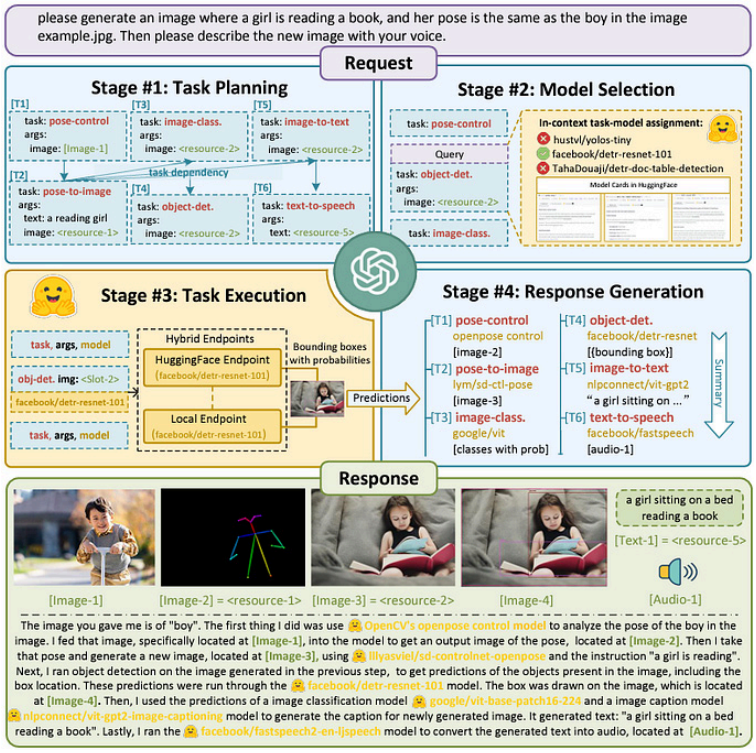


Figure 9.7 – HuggingGPT process (https://arxiv.org/pdf/2303.17580)

Task planning

In the first step, **task planning**, the LLM must understand the task and break it down into subtasks. In the real world, user requests can be complex and their intentions intricate, requiring task decomposition. This is because a single model may not be capable of solving the entire task; instead, multiple models might be necessary to address different aspects. An LLM then needs to decompose the task into a series of subtasks and understand the dependency between these tasks and in what order they should be executed. This is conducted by creating a specific prompt.

To standardize the system, the authors of HuggingGPT used a set of specific instructions. An LLM must then adhere to these specifications in order to conduct task planning. They designed a standardized template for tasks and instructed the LLM to conduct task parsing through slot filling. The LLM is guided to fill this template using slot filling, allowing for the consistent parsing and execution of subtasks. There are four slots that the template must fill:

- **Task ID**: The model provides a unique identifier for each task. This ID is used to identify both the task and dependent tasks, as well as all the resources that are generated.
- **Task type**: This slot includes the task type; each task can be of various types (language, visual, video, audio, and so on).
- **Task dependencies**: This slot defines the prerequisites for each task (the model only launches a task if all its prerequisites are complete).
- **Task arguments**: This slot contains all the arguments that are required for the execution of a task (from text to images or other resources). These contents can be derived from the user's query or from the results of other tasks.

Task	Args
Text-cls	text
Token-cls	text
Text2text-generation	text
Summarization	text
Translation	text
Question-answering	text
Conversational	text
Text-generation	text
Tabular-cls	text

Table 1: NLP tasks.

Task	Args
Image-to-text	image
Text-to-image	image
VQA	text + image
Segmentation	image
DQA	text + image
Image-cls	image
Image-to-image	image
Object-detection	image
Controlnet-sd	image

Table 2: CV tasks.

Task	Args
Text-to-speech	text
Audio-cls	audio
ASR	audio
Audio-to-audio	audio

Table 3: Audio tasks.

Task	Args
Text-to-video	text
Video-cls	video

Table 4: Video tasks.

Figure 9.8 – HuggingGPT type of task (https://arxiv.org/pdf/2303.17580)

The authors use demonstrations to direct the model to perform a task (such as image-to-text, summarization, and so on). As we saw in ***Chapter 3***, adding demonstrations allows the model to map the task (few-shot prompting and in-context learning). These demonstrations tell the model how it should divide the task, in what order, and whether there are dependencies. In addition, to support complex tasks, the authors include chat logs (previous discussions that were conducted with the user) as a kind of tool. This way, the model can be aware if additional resources or requests have been indicated that can help with the task.

The prompt provides all the information needed for the LLM. In the prompt, we provide instructions on its task (planning the task breakdown), where to retrieve information, examples of how it should perform the task, and what output we expect.

Task Planning	Prompt	
	#1 Task Planning Stage - The AI assistant can parse user input to several tasks: [{"task": task, "id", task_id, "dep": dependency_task_ids, "args": {"text": text, "image": URL, "audio": URL, "video": URL}}]. The "dep" field denotes the id of the previous task which generates a new resource that the current task relies on. A special tag "<resource>-task_id" refers to the generated text image, audio and video in the dependency task with id as task_id. The task MUST be selected from the following options: {{ Available Task List }}. There is a logical relationship between tasks, please note their order. If the user input can't be parsed, you need to reply empty JSON. Here are several cases for your reference: {{ Demonstrations }}. The chat history is recorded as {{ Chat Logs }}. From the chat logs, you can find the path of the user-mentioned resources for your task planning.	
	Demonstrations	
	Look at /exp1.jpg, Can you tell me how many objects in the picture?	[{"task": "image-to-text", "id": 0, "dep": [-1], "args": {"image": "/exp1.jpg" }}, {"task": "object-detection", "id": 0, "dep": [-1], "args": {"image": "/exp1.jpg" }}]
	In /exp2.jpg, what's the animal and what's it doing?	[{"task": "image-to-text", "id": 0, "dep":[-1], "args": {"image": "/exp2.jpg" }}, {"task":"image-classification", "id": 1, "dep": [-1], "args": {"image": "/exp2.jpg" }}, {"task":"object-detection", "id": 2, "dep": [-1], "args": {"image": "/exp2.jpg" }}, {"task": "visual-question-answering", "id": 3, "dep":[-1], "args": {"text": "What's the animal doing?", "image": "/exp2.jpg" }}]
	Given an image /exp3.jpg, first generate a hed image, then based on the hed image and a prompt: a girl is reading a book, you need to reply with a new image.	[{"task": "image-to-text", "id": 0, "dep": [-1], "args": {"image": "/examples/boy.jpg" }}, {"task": "openpose-control", "id": 1, "dep": [-1], "args": {"image": "/examples/boy.jpg" }}, {"task": "openpose-text-to-image", "id": 2, "dep": [1], "args": {"text": "a girl is reading a book", "image": "<resource>-1" }}]

Figure 9.9 – Details of the prompt design in HuggingGPT (https://arxiv.org/pdf/2303.17580)

Model selection

After planning the task, the model proceeds to select appropriate models for the task, or **model selection**. Once we have a list of subtasks, we need to choose the appropriate model. This is possible because we have descriptions of the models and what they do. The authors of this work have collected descriptions of expert models from the ML community (e.g., Hugging Face). In fact, on Hugging Face, it is often the model's developers themselves who describe the model in terms of functionality, architecture, supported languages and domains, licensing, and so on.

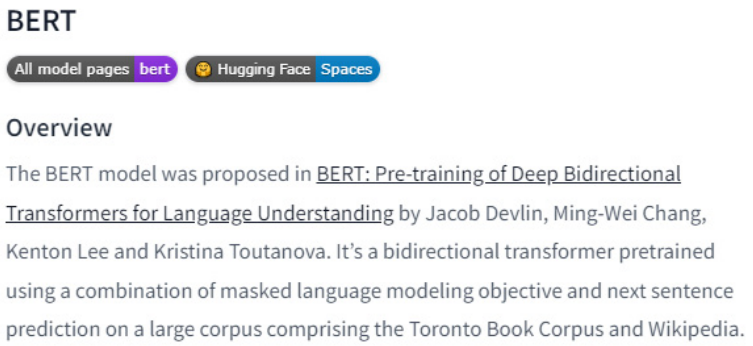


Figure 9.10 – Screenshot of an example of the description of a model on Hugging Face (https://huggingface.co/docs/transformers/model_doc/bert)

Model assignment is thus formulated as a single-choice model, in which an LLM must choose which model is the best among those available given a particular context. Then, considering the user's requirements and the context, an LLM can choose which expert model is best suited to perform the task. Of course, there is a limit to the context length, and you cannot enter all the model descriptions without exceeding this length. To address this, the HuggingGPT system applies a two-stage filtering and ranking process. First, models are filtered based on the task type identified during task planning (e.g., language, vision, or audio). Only models that are relevant to the specific subtask type are retained, narrowing down the pool significantly. Among the filtered models, the system sorts them based on the number of downloads, which acts as a proxy for quality, reliability, and community trust. The assumption is that widely used models are more likely to perform well. Finally, the system selects the top-k model descriptions (where k is a configurable hyperparameter) and includes them in the prompt. The LLM then performs single-choice model selection, evaluating the context and user requirements to choose the most appropriate model from the shortlist. This strategy offers a balanced trade-off: it keeps the prompt within manageable token limits while still allowing the LLM enough options to make an informed and effective model selection.

	Prompt
Model Selection	#2 Model Selection Stage - Given the user request and the call command, the AI assistant helps the user to select a suitable model from a list of models to process the user request. The AI assistant merely outputs the model id of the most appropriate model. The output must be in a strict JSON format: "id": "id", "reason": "your detail reason for the choice". We have a list of models for you to choose from {{ Candidate Models }}. Please select one model from the list.
	Candidate Models
	{"model_id": model id #1, "metadata": meta-infos #1, "description": description of model #1} {"model_id": model id #2, "metadata": meta-infos #2, "description": description of model #2} [Select top-K candidate models with the same task type as the current task] {"model_id": model id #K, "metadata": meta-infos #K, "description": description of model #K}

Figure 9.11 – Details of the prompt design in HuggingGPT for model selection (https://arxiv.org/pdf/2303.17580)

Model execution

Once a specific model has been assigned to a specific task, the model must be executed. Note that these models are used only in inference. These models are used through the Hugging Face API. To speed up execution, HuggingGPT uses hybrid inference endpoints. The selected model takes the task arguments as input and then sends the results back to the language model (ChatGPT). Moreover, if the model has no resource dependencies, its inference can be parallelized. In other words, tasks that are not dependent on each other can be executed simultaneously. Otherwise, the system takes into account how much the output of one model and the

input of another are connected (e.g., if one task must have the output of another subtask in order to be carried out). To perform inference, HuggingGPT uses hybrid inference endpoints, primarily relying on Hugging Face APIs. When models are available and functional via these APIs, the system executes them remotely. However, if API endpoints are unavailable or slow or face network issues, local inference is used as a fallback. This hybrid setup ensures flexibility and robustness in execution.

The authors note: "*Despite HuggingGPT's ability to develop the task order through task planning, it can still be challenging to effectively manage resource dependencies between tasks in the task execution stage*." To solve this problem, the authors simply used a unique symbol, **<resource>**, to handle the dependencies. **<resource>** is a special token that represents the resource required for a task (this matches the task identifier), and if the required task is completed, the token is replaced with the resource.

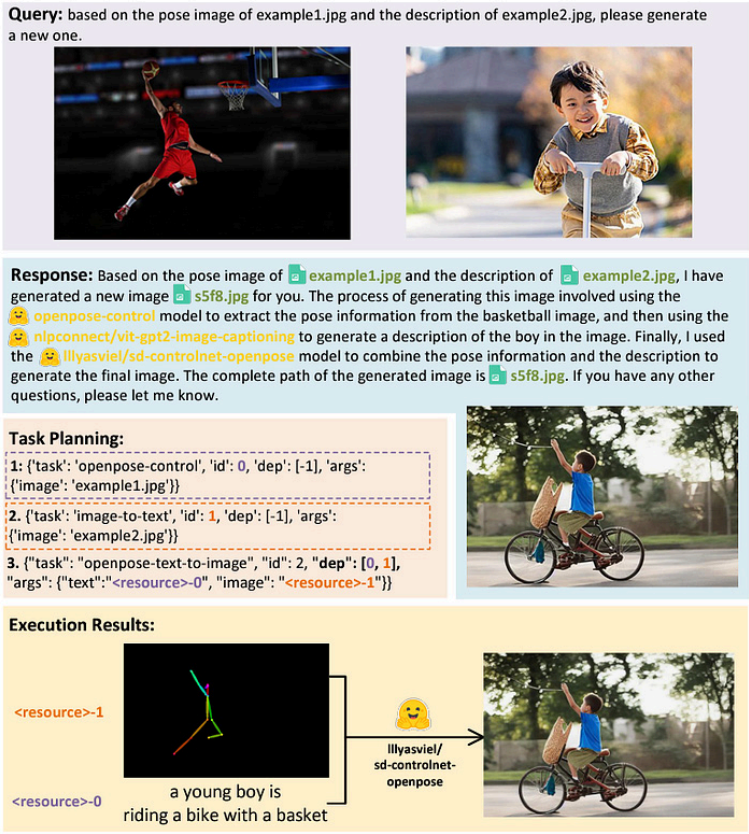


Figure 9.12 – Model execution (https://arxiv.org/pdf/2303.17580)

Response generation

Once all the tasks are executed, the response must be generated. HuggingGPT integrates all the information that was obtained in the previous steps (task planning, model selection, and task execution) into a kind of concise summary (the tasks, the models used, and the results of the models). Note that the model integrates results of several other models, especially those obtained by inference and that may be of different formats. These results are presented in a structured format (as in, bounding boxes, probabilities, and so on), and HuggingGPT takes these results and transforms them into natural language to respond to a user. So, HuggingGPT not only gets results for the task but also responds to the user in a human-friendly way.

	Prompt
Response Generation	#4 Response Generation Stage - With the input and the inference results, the AI assistant needs to describe the process and results. The previous stages can be formed as - User Input: {{ User Input }}, Task Planning: {{ Tasks }}, Model Selection: {{ Model Assignment }}, Task Execution: {{ Predictions }}. You must first answer the user's request in a straightforward manner. Then describe the task process and show your analysis and model inference results to the user in the first person. If inference results contain a file path, must tell the user the complete file path. If there is nothing in the results, please tell me you can't make it.

Figure 9.13 – Response generation (https://arxiv.org/pdf/2303.17580)

Qualitatively, we can see that the model is capable of solving several tasks. Thus, the model is able to divide the task into various subtasks, choose appropriate models, retrieve the results, and integrate them efficiently. For example, the model can do image captioning, pose generation, and even pose conditional image generation tasks. Not only that but the tasks can be multimodal (such as text-to-video generation, adding audio to a video, and so on). One of the most interesting aspects is that all of this is conducted without any additional LLM training. In fact, everything is done in inference (for both LLMs and models in inference). The advantage is that you can integrate additional models for additional tasks without any training; you only need to add a functional description of the new models.

For example, in this case, we can see the execution of a multimodal task (text, video, and audio). The model is asked to perform two tasks: generate a video from a description and dub the video. The model performs these two actions in parallel. In the bottom part of the following figure, the model must instead perform the two tasks in series: the model first generates text from the image and then generates audio.

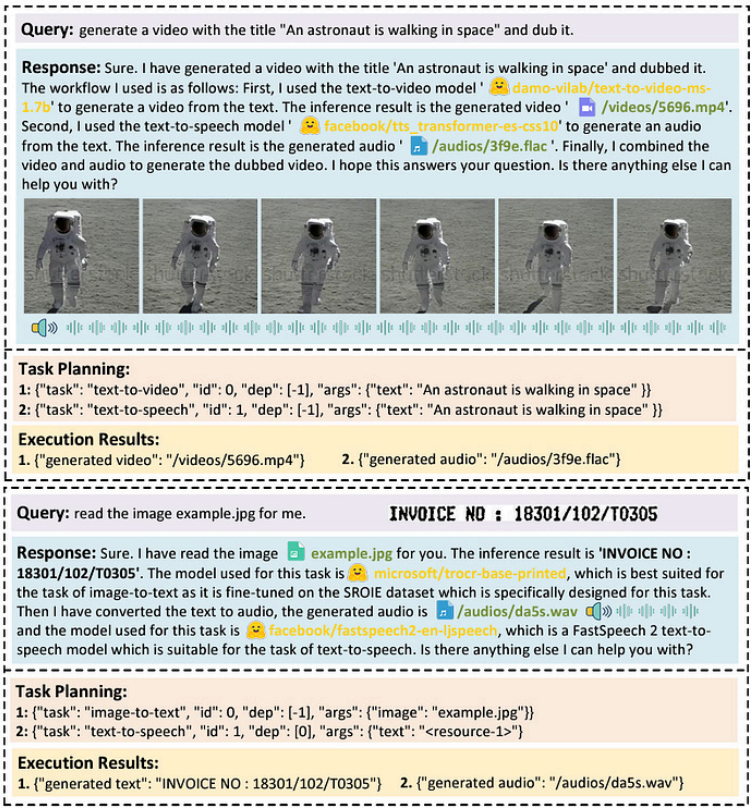


Figure 9.14 – Qualitative analysis of multi-model cooperation on video and audio modalities (https://arxiv.org/pdf/2303.17580)

The authors of the study also explore more complex tasks where an LLM must organize the cooperation of multiple models to succeed in solving the task. HuggingGPT can organize the cooperation of multiple models through the task planning step. The results show that HuggingGPT can

cope with complex tasks in a multi-round conversation scenario (where the user divides their requests into several rounds). Moreover, the model can solve complex tasks by assigning an expert model to each task. For example, "*Describe the image in as much detail as possible*" requires the model to solve five tasks (image caption, image classification, object detection, segmentation, and visual question-answering tasks). These five tasks are not solved by one model but by five different models that are called and executed. Each of these models then provides information that must be integrated into a detailed answer. These models work in parallel in inference and then the final information is merged.



Figure 9.15 – Case study on complex tasks (https://arxiv.org/pdf/2303.17580)

HuggingGPT limitations

However, some limitations remain:

- **Efficiency**: HuggingGPT requires multiple calls from an LLM; this occurs in three of the four process steps (task planning, model selection, and response generation). These interactions are expensive and can lead to response latency and degradation of the user experience. In addition, closed-source models (GPT-3.5 and GPT-4) were used in the original article, leading to additional costs. Technically, the same approach could have been carried out with models that are open source.

- **Planning**: Planning depends on the capabilities of the LLM. Obviously, the more capable an LLM, the better the system's capabilities, but an LLM has limited reasoning capabilities, so planning may not always be optimal or feasible. You could then test different LLMs or use LLMs that are fine-tuned to create an efficient plan or models that have been fine-tuned to the reasoning chain.
- **Context lengths**: The context length of a model has a definite limit, and for complex tasks, this is a problem. In the original article, the authors note that 32K for some tasks is enough (especially if several models are connected). The solution, then, may be to use models with a longer context length. To date, though, it seems that models don't use long context efficiently. Another solution might be to use summarization.
- **Instability**: This stems from the stochastic nature of the LLM. Although LLMs are trained to generate text, and in this case we provide context, the model can ignore context and hallucinate. The authors of the article note that the model may fail to conform to instructions or give incorrect answers during the prediction. This generates program flow errors or incorrect answers. Hallucinations are still an open problem for LLMs, but there are strategies to mitigate them.

HuggingGPT, then, is a system capable of solving complex tasks by orchestrating different expert models using the language as an interface. The LLM acts here only as a controller and manager of the various AI models. Its only tasks are to orchestrate the models and then generate a response. The model then generates a plan, selects the models, and then integrates the results into the final response. By itself, the LLM does not perform any tasks but demands resolution from the various expert models. All this is conducted in inference without any training. The user then provides their question, and the system conducts the process and then responds in natural language, thus making the interaction human-friendly and fluid.

In the following subsections, we will examine various models designed to overcome the limitations of HuggingGPT or address critical challenges in other specialized domains. Through these explorations, you will gain insight into different strategies and learn how these agents can be applied to real-world scenarios.

ChemCrow

We previously saw HuggingGPT as a system that orchestrates different tools (models), acting as a generalist model for general tasks. In this subsection, we want to discuss a similar system applied to a specialized field. ChemCrow (Bran, 2023) follows a similar design philosophy to HuggingGPT, but applies it to a specialized field—chemistry.

The limitation of generalist LLMs is that they have generalist knowledge and therefore are neither specialized for a field nor updated with the latest information. This can be a problematic limitation for many application fields (especially specialized ones such as science, finance, and healthcare). In addition, LLMs conduct calculations using a bag of heuristics and not by a rigorous process. For fields such as chemistry, this is a problem, so it is natural to think about extending the models' capabilities with external tools. External tools then provide the exact answer and compensate for the deficiencies of LLMs in specific domains. Thus, having an integration of an LLM with several tools can allow an LLM to be

used even in fields where its inherent characteristics constitute a limitation to its applicability.

One field that can benefit from the use of LLMs is scientific research. On the one hand, LLMs have shown some ability to understand chemistry, and on the other hand, there are many specialized models for chemistry, or at least for specific applications. Many of these tools have been developed by the open source community and are accessible through APIs. Nevertheless, integrating these tools is not easy and requires expertise in computational coding, which is often not among the skills of chemistry researchers. Inspired by previous work, the authors of this study (Bran, 2023) proposed what they call an LLM-powered chemistry engine (ChemCrow) to "*streamline the reasoning process for various common chemical tasks across areas such as drug and materials design and synthesis*." ChemCrow is very similar to what we have seen with HuggingGPT, in which we have a central LLM (GPT-4) that orchestrates a number of tools (in this case, highly specialized for chemistry). The central LLM is prompted with specific instructions and information in order to perform the tasks specifically and respond in a specific format. To guide the LLM's reasoning and tool use, ChemCrow adopts a structured prompting format known as Thought, Action, Action Input, and Observation, to prompt the model to reason about the task (and its current state), how the current state relates to the final goal, and how to plan the next steps:

- **Thought**: The model reflects on the current problem, considers its progress, and outlines reasoning toward the final goal
- **Action**: It selects the appropriate tool to use next (e.g., a molecule generator or a reaction predictor)
- **Action input**: It specifies what input should be sent to the chosen tool
- **Observation**: It records the tool's output, which is then incorporated into the next reasoning cycle

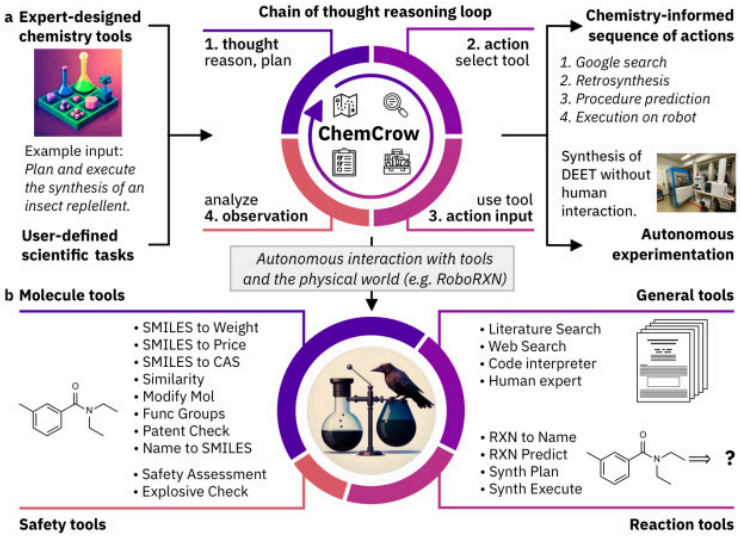


Figure 9.16 – Overview of ChemCrow (https://arxiv.org/pdf/2304.05376)

So, in this system, the model proceeds with a Thought step (which can be thought of as action planning) and uses a tool and an input to this tool (selecting and using the model). The model gets the results, observes them, and conducts a Thought step again until the answer is reached. The process is similar to what we saw in the previous section, but there is a greater emphasis on reasoning and a specialization of the model. Also, among the tools are not only models but also the ability to search the internet or the literature; the model can also run code. So, we also have an

extension of the capabilities and flexibility of the system. Thus, the authors of the study see this system as a kind of researcher's assistant to perform chemical tasks.

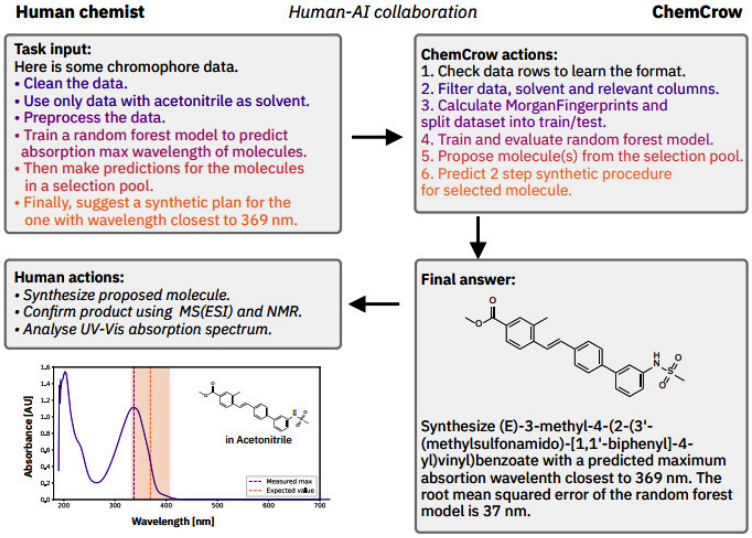


Figure 9.17 – Human/model interaction leading to the discovery of a novel molecule (https://arxiv.org/pdf/2304.05376)

So, the idea is to combine LLM reasoning skills with expert knowledge and chemical computational tools. The results show that similar approaches can lead to real-world applications in specific fields, such as chemistry.

SwiftDossier

SwiftDossier is a notable example of applying agent-based systems in the scientific and healthcare domains, with a particular focus on addressing one of the most critical challenges in these areas: hallucinations. In fields such as medicine and pharmaceuticals, hallucinated outputs—that is, confident but false or unverifiable information—can lead to serious legal, ethical, and safety risks. An LLM has a huge memory but generates text stochastically, without obviously verifying its sources. This is problematic for the pharmaceutical industry or potential use in medicine. To solve this problem in SwiftDossier, RAGs and LLM-powered agents are used to force model generation. Instead of relying solely on the LLM's internal knowledge—which is vast but generated stochastically and without source verification—the system forces the model to ground its responses in external, reliable data sources. The system uses a different set of tools to be able to answer different questions: scientific articles, internet access, databases, and other ML models. Using this set of tools, an LLM can succeed in generating reports and minimize the risk of hallucination.

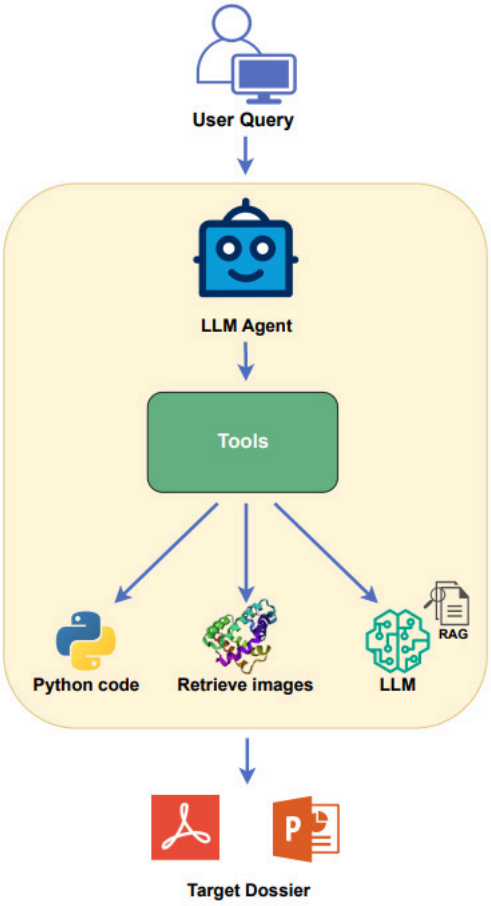


Figure 9.18 – SwiftDossier architecture (https://arxiv.org/pdf/2409.15817)

ChemAgent

In the two examples seen previously, we have an agent to which tools are added to make up for the knowledge deficiencies of a generalist LLM. In other words, we try to make up for the shortcomings of an LLM by using either external information or tools to conduct operations. Moreover, if the task itself is complex, several approaches try to decompose it into more manageable subtasks. An agent first produces a schedule and then executes the various subtasks, thus combining reasoning and execution. Despite all this, an LLM may still generate errors, especially in complex domains such as chemistry.

LLMs, while powerful general-purpose tools, face several challenges in the chemistry domain, where tasks require precise reasoning, accurate calculations, and deep domain knowledge. These challenges arise due to the limitations in how LLMs generate text and code, and they become more pronounced in scientific applications where small errors can lead to significant inaccuracies:

- **Struggles with domain-specific formulas**: LLMs may misinterpret or incorrectly apply specialized chemical equations or notation, especially when the required formulas are not commonly found in general training data
- **Incorrect intermediate reasoning steps**: In complex, multi-step tasks (e.g., synthesis planning or property prediction), an error in just one step can cascade and lead to faulty final outputs
- **Errors in code generation**: When combining textual reasoning with code (typically Python), LLMs often hallucinate functions, use incorrect libraries, produce syntax errors, or generate code that fails to exe-

cute—especially for scientific calculations that require precise library calls and numerical stability

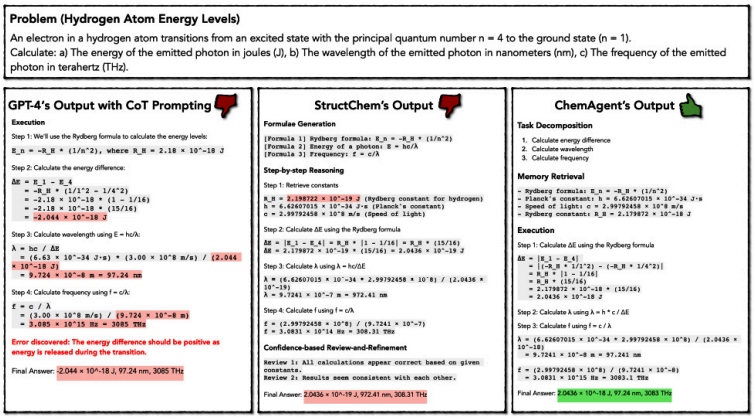


Figure 9.19 – Examples of LLM failure in chemistry domain (https://arxiv.org/pdf/2501.06590)

Human beings, unlike LLMs, learn from their past experiences and mistakes. For LLMs, it is not possible to learn after the end of pre-training (fine-tuning is an expensive approach and cannot be used repeatedly), so continual learning remains an open problem of AI. Humans, on the other hand, can remember strategies used for similar problems; once they encounter new problems, they learn new strategies that can be used in the future. Therefore, in ChemAgent, the authors try to find a way to simulate this process. They propose a dynamic library that allows iterative problem-solving to be facilitated by continuously updating and refining its content. The library serves as a repository for decomposed chemical tasks. In other words, a task is broken down into various subtasks and then the solutions are saved in the library for future use. Once a new task arrives, the library is updated with the new subtasks and corresponding solutions, keeping the library relevant and improving its usefulness over time. Inspired by human cognition, the system has three different memory components: planning memory (high-level strategies), execution memory (specific task solutions), and knowledge memory (fundamental chemistry principles). These memory components are stored externally, allowing the system to find the information again when needed, and are dynamically updated.

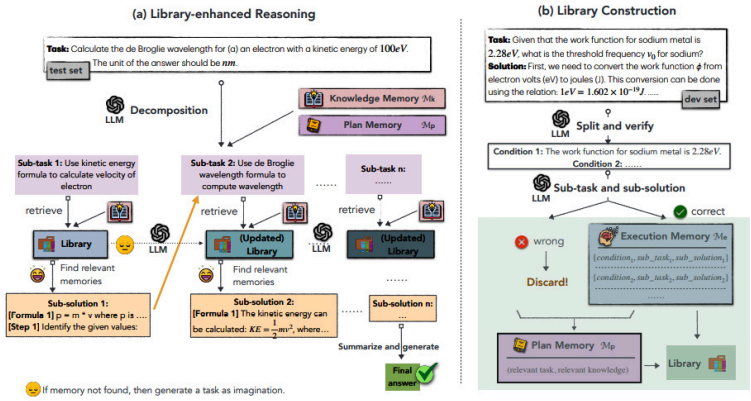


Figure 9.20 – ChemAgent framework (https://arxiv.org/pdf/2501.06590)

ChemAgent thus doesn't just passively use what it finds in memory but rather allows the system to update the memory dynamically. It also uses memory partitioning to improve the various stages of problem-solving. ChemAgent divides the process into planning and execution (to which it associates a specific memory for each step) and adds memory that func-

tions as a reference for fundamental chemistry principles and formulas. When a problem occurs, it is divided into a series of subtasks, which are solved, and these solutions are saved in memory.

Multi-agent for law

Another area that could benefit from the use of agents is the legal sector. Legal services are essential to protect citizens' rights, but they can be particularly expensive and there are not always enough lawyers. Moreover, fair judgment is a fundamental right, but human beings also exhibit bias. Using agents in this field could revolutionize legal services by lowering costs and allowing more equitable access. In the legal field, hallucinations are particularly problematic and should be, if not eliminated, reduced as much as possible. Hallucinations arise from both the stochastic nature of the models and the quality of the data with which they are trained. Therefore, action must be taken on two axes in order to mitigate the phenomenon.

In this subsection, we want to present two law-focused approaches to present some interesting elements that have been used. Again, the principle is the same: everything revolves around a central element, which is an LLM. For example, Chatlaw focuses on data quality to mitigate the risk of LLM hallucination. Also, to make the most of the quality dataset the authors have collected, they use a knowledge graph. In addition, instead of using a single agent, they use a multi-agent system. Using multiple agents allows the system to simulate different areas of expertise, thanks to the flexibility of prompts when interacting with LLMs. The use of multi-agents makes it possible to emulate the process within a law firm. The authors developed a protocol to allow effective collaboration among agents: "*four independent intelligent agent roles responsible for initial information gathering, in-depth material research, legal advice, and final consultation report writing*." In this way, the process is more thorough. Again, they used only one LLM for the whole system (the authors used GPT-4).

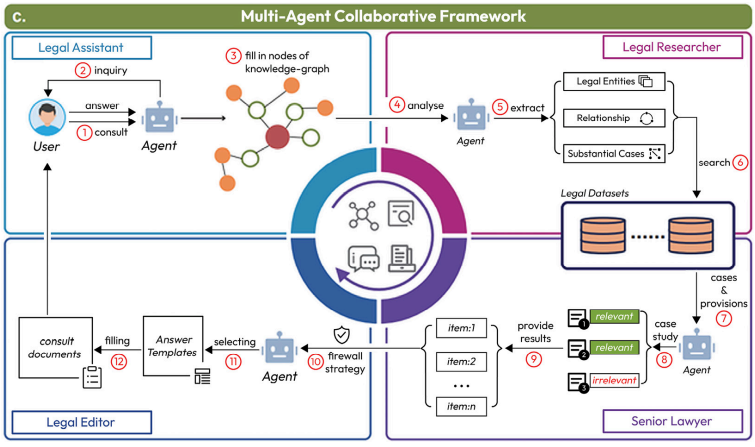


Figure 9.21 – Chatlaw, a multi-agent collaboration (https://arxiv.org/pdf/2306.16092v2)

Another interesting approach is one in which the authors (Hamilton, 2023; https://arxiv.org/pdf/2301.05327) mimic the judgment of a court using an LLM. Here, too, a multi-agent system is used, in which each agent represents a judge. Each judge produces an opinion and then a majority opinion is obtained. So, when a case is sent to nine judges, the system receives nine opinions, and then it produces a single opinion. This

approach then relies on conducting nine evaluations in parallel and the consistency of these evaluations (the majority vote wins).

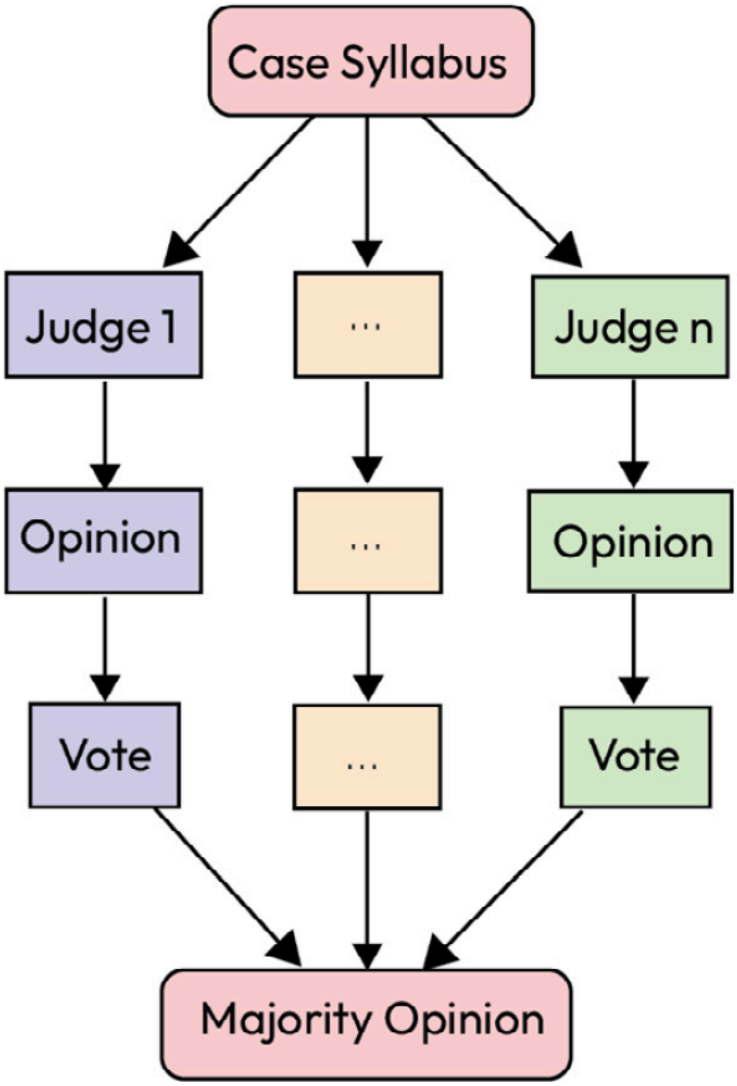


Figure 9.22 – Multi-judge system (https://arxiv.org/pdf/2301.05327)

This work shows how to leverage an LLM to create multiple agents that work together to be able to mitigate hallucinations. The authors are further evidence of the flexibility that can be achieved by using an LLM as the center of the system. A limitation of this study is the use of homogeneous judges (it would be better to build the ensemble with different models, to avoid the various judges having the same bias), risking repetitive opinions.

Multi-agent for healthcare applications

Interdisciplinary research is complex and usually requires teams composed of researchers with different areas of expertise. Typically, scientific research is conducted by teams where each researcher deals with a particular aspect and masters different techniques. For example, AlphaFold 2 is the product of 34 researchers with different expertise (computer science, bioinformatics, and structural biology). Obviously, recruiting large teams of experts takes time (and it is not always easy to find people with the right expertise) and is expensive. Only a few institutions and companies can afford the most ambitious projects. Recently created LLMs, though, have increasingly broad knowledge of scientific topics, and we saw previously that this knowledge can be connected to the use of tools. ChemCrow is an example of how to solve a chemical problem, but it can-

not tackle an open-ended, interdisciplinary research problem. Recently, efforts have been made to solve this problem by creating pipelines that can handle the end-to-end process. For example, an AI scientist (Lu, 2024) carries out a process that starts with conceptualizing an idea and ends with writing a scientific paper on ML. The AI scientist is given a broad research direction, produces an idea, conducts the literature search, plans and executes experiments, writes a manuscript, and, finally, proofreads it. All this is done by an LLM-like agent that is connected to tools and proceeds sequentially.

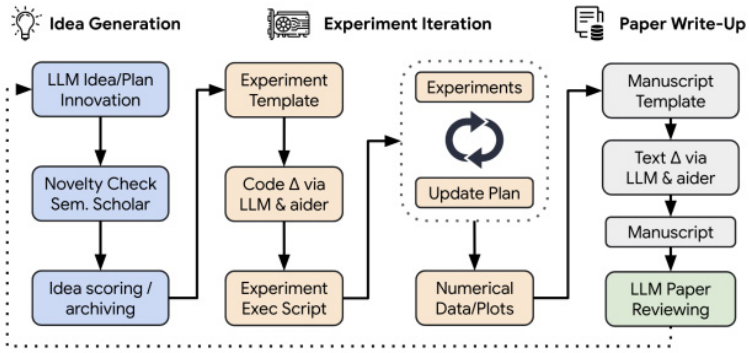


Figure 9.23 – Illustration of the AI scientist process (https://arxiv.org/pdf/2408.06292)

Other works also show similar processes, but they are still localized to specific fields and linear processes. For scientific research, we want to find ways to combine different expertise. Swanson (2024), therefore, proposes a Virtual Lab for human-AI collaboration with the purpose of performing interdisciplinary science on complex questions. In the Virtual Lab, a human leads a set of interdisciplinary agents to manage a complex process. The different agents have different expertise and are run by an LLM. Each of these agents interacts with both other agents and a human being. In this way, the authors of the study build a flexible architecture. Here, the human being provides guidance to the agents, while the agents are the ones that decide on search directions and design solutions to the problem. Each agent is controlled by a prompt (which contains information about the role, expertise, goal, and available tools) provided to an LLM (GPT-4 in the article). The Virtual Lab then conducts the research in group or individual meetings.

The human provides the question and agenda to start the discussion. In team meetings, agents discuss the research question and work together toward the global goal. In individual meetings, a single agent has to solve a task (such as writing code) and the agent works alone or together with another agent who provides critical feedback. With a series of global and individual meetings, the team solves a research question.

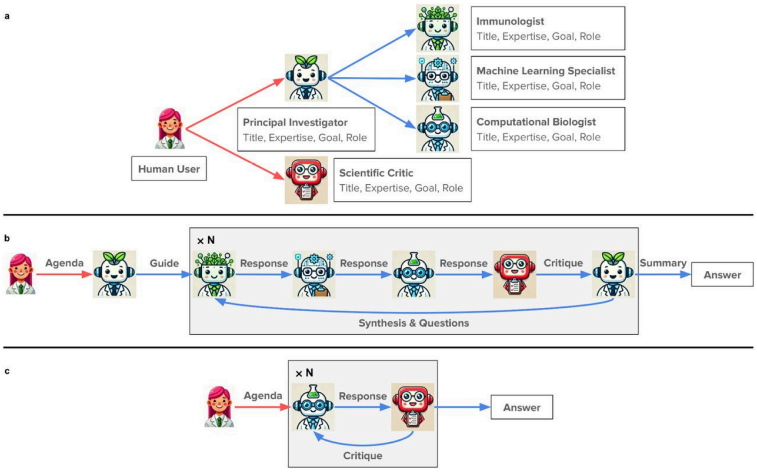


Figure 9.24 – Architecture of a Virtual Lab (https://www.biorxiv.org/content/10.1101/2024.11.11.623004v1.full)

In the Virtual Lab, there is a **Principal Investigator** (**PI**) whose purpose is to maximize the impact of the research, and who automatically creates a set of appropriate scientist agents (biologists or computer scientists) for the project (based on the project description provided by the PI). The PI defines each agent's role, expertise, and goal in a prompt. In addition, there may be an agent dedicated to project critique. After that, the meetings begin. Each meeting follows a set of inputs organized into a structure: agenda (a description of what is to be discussed), agenda questions (a set of questions to be answered in the meeting), agenda rules (a set of optional rules to make the meeting smoother), summaries (optional summaries of previous meetings), contexts (additional information that can help the meeting), and rounds (the number of rounds of discussion to prevent the discussion from continuing endlessly). In the team meeting, all agents participate in the discussion, the human writes the agenda (optionally, along with rules and questions), and different rounds of discussion follow. The PI starts and then each of the scientist agents (plus the critic agent) gives their thoughts on the discussion. At the end, the PI summarizes the points posed by the agents, makes a decision on the agents' inputs, and asks follow-up questions. After the various rounds, the PI writes a final summary that the human can read.

In individual meetings, the human provides the agenda and selects the agent, and the agent performs the task (there may, in addition, be the critic agent, who provides critiques). After a series of rounds between the agent and critic, the agent provides the response. In addition, parallel meetings may be conducted, in which multiple agents perform the same task, and in a final meeting with the PI, the final answer is arrived at.

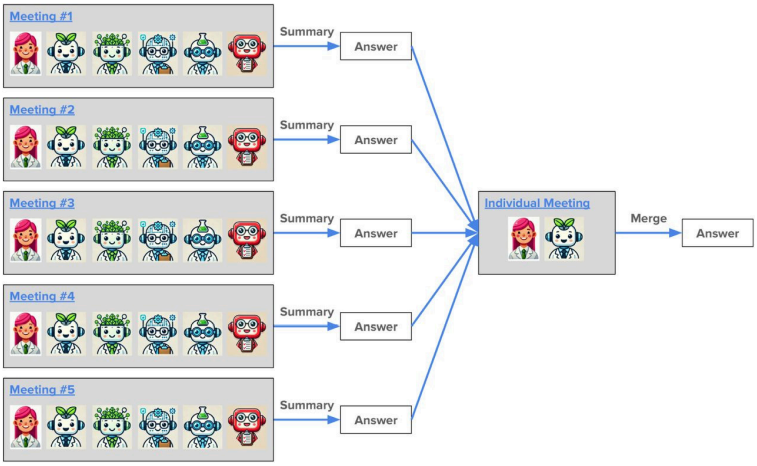


Figure 9.25 – Virtual Lab parallel meetings (https://www.biorxiv.org/content/10.1101/2024.11.11.623004v1.full)

In this way, the authors have created a flexible framework that combines heterogeneous agents that work in both single and collaborative settings. It should be noted that in this approach, there is a human in the loop; that is, a human being is at the center of the system and actively collaborates with the AI. This process mimics (though, of course, in a simplified way) the work and decision-making process of a human team when it has to solve a complex problem. To test the usefulness of this work, the authors tested the Virtual Lab on designing antibodies or nanobodies that can bind to the spike protein of the KP.3 variant of SARS-CoV-2. This is a complex problem because SARS-CoV-2 evolves rapidly, so a fast system must be found to design antibodies that can block it. The Virtual Lab started by creating a team that could tackle the problem (the PI created the right team of researchers for the problem). In a team meeting, the direction of the project was described and the principal details were discussed. There was then a team meeting about which tools could be used and were selected, as well as a series of individual meetings where the researchers used the various tools to create the antibody design workflow. In a meeting with the PI, the workflow was defined.

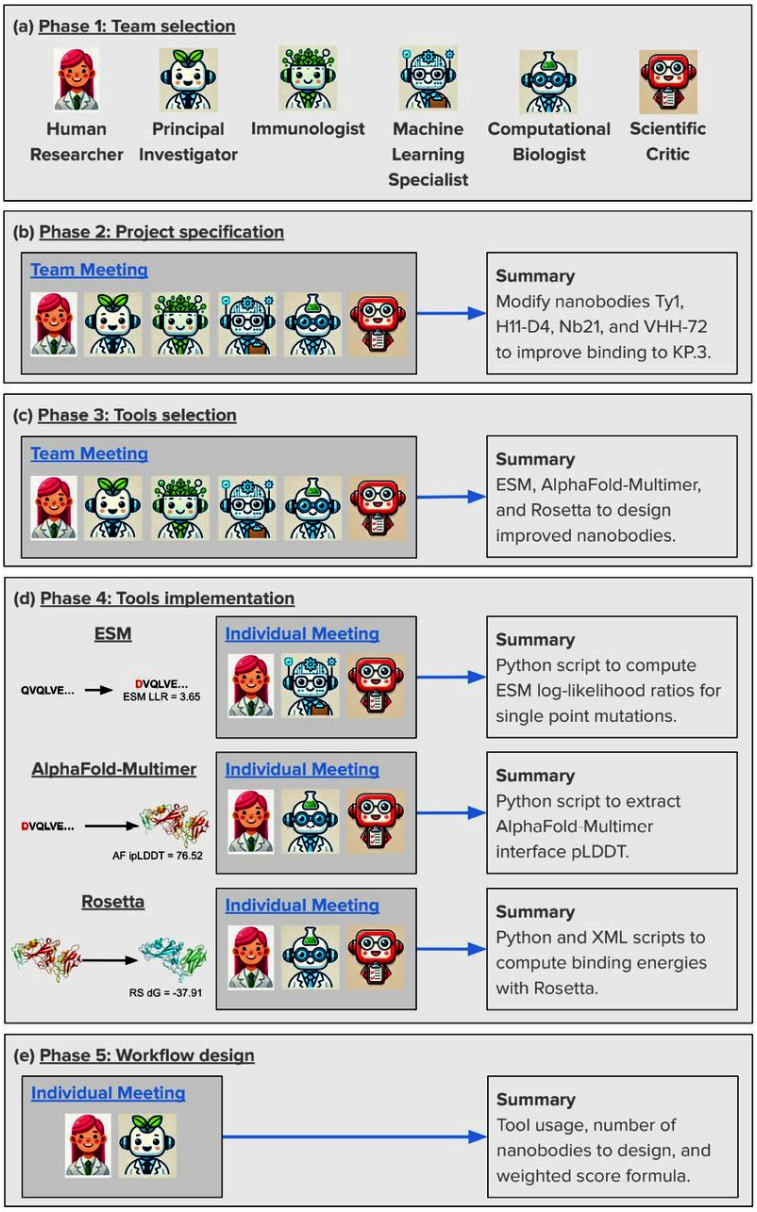


Figure 9.26 – Virtual Lab for antibody design (https://www.biorxiv.org/content/10.1101/2024.11.11.623004v1.full)

The Virtual Lab managed to design antibodies that they then validated experimentally. The system managed to create a complex workflow that used serial models to design antibodies (thus solving a real and complex problem). Building this would usually require a multidisciplinary team because the problem needs to be solved with different expertise. Thus, having agents with different expertise allows the problem to be discussed from different angles, to which a fundamental element of scientific research (critique) is added. This is done through a series of meetings, where the AI is a partner to the human being. What we see here is the creation of a multi-agent and heterogeneous system with multiple rounds of meetings (group and individual) to create a system that is flexible and sophisticated at the same time.

There are still limitations at this stage. For example, the models have knowledge up to a certain cut-off point, so they may not be aware of the latest published tools and could thus suggest old models (or ones that have problems in implementation). The solution to this problem might be to use RAG or an internet search. Another limitation is that the system is not exactly self-contained; it comes with both an agenda and a set of prompts that have been carefully designed. In this system, human beings are still involved and must provide guidance. Without guidance, the AI models may give vague answers or not make decisions unless specifically requested. Also, sometimes they do not accomplish the task or they deviate from what they are supposed to do. In any case, this system is flexible and can be applied agnostically to many other problems.

Combining different expertise with human feedback seems to be the key to better results. In a similar vein, Agent Laboratory is designed to generate an entire research workflow (from literature review and experimentation to report writing), all from an initial human-provided research idea. In this system, the process begins with the collection and analysis of relevant papers, followed by collaborative planning and data preparation, a series of experiments, and report generation. The process can be divided into three stages:

- **Literature review**: In this stage, articles are collected for the given research idea. A PhD agent utilizes the arXiv API to retrieve related papers, synthesizes them, and provides insights. This agent uses search APIs, summarization models, and bibliographic management systems as tools. The process is iterated until it reaches a certain number of relevant articles.
- **Experimentation**: The first step is plan formulation, where a plan is generated based on the literature review and the research goal. At this stage, the PhD and Postdoc agents collaborate and discuss how to achieve the goals, generating a plan that defines which ML models to implement, which datasets to use, and other necessary experimental steps. Once the plan is finalized, the data preparation phase begins, during which the code for data preparation is generated based on the defined plan. An ML engineer agent has access to Hugging Face datasets, and the code is then compiled and submitted. During the running experiments phase, the ML engineer agent executes the experimental plan. At this stage, the code is generated, tested, and refined. The results are then interpreted. At the end of this phase, the PhD and

Postdoc agents discuss the results. If they agree on the validity of the findings, they submit the results, which will serve as the basis of the report.

- **Report writing**: In the report writing phase, the PhD and professor agents synthesize the research findings into a comprehensive academic report. Starting with an initial scaffold (abstract, introduction, background, related work, methods, experimental setup, results, and discussion), they begin generating the text (which is written in LaTeX for easy revision and correction). During writing, the system accesses the literature and iteratively corrects the article for accuracy, clarity, and alignment with the research goals. Finally, a sort of paper review is conducted to ensure the article is correct. Note that during this process, the system receives feedback from humans.

The key features of this system are that the agents perform repetitive tasks (e.g., literature searches and coding) autonomously but allow for human input where creativity or judgment is essential. The agents communicate intermediate results with each other to ensure cohesion among the parties. At each stage, there is iterative improvement through reflection and feedback.

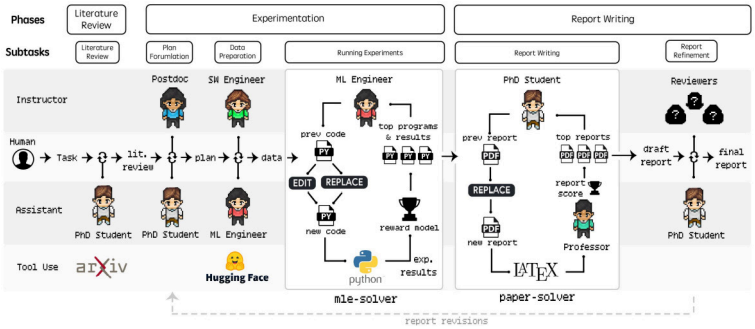


Figure 9.27 – Agent Laboratory workflow (https://arxiv.org/pdf/2501.04227)

Agent Laboratory is designed to explore ideas quickly and help researchers in being able to explore multiple lines of research at the same time. The structure of Agent Laboratory allows it to conduct the entire workflow from an idea suggested by a human researcher. In this work, they focus on not only the accuracy of the results but also on trying to find a more efficient way of solving the task (previous work required too much computational cost).

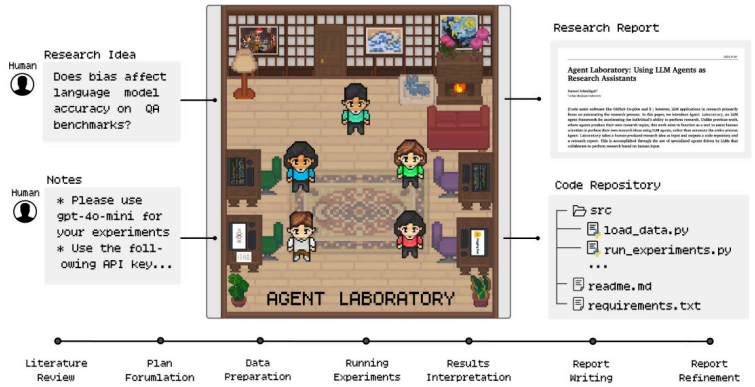


Figure 9.28 – Agent Laboratory scheme (https://arxiv.org/pdf/2501.04227)

The authors point out that incorporating human feedback at various stages significantly improved the quality of the research outputs. Furthermore, they state that ML code generated by Agent Laboratory

achieved performance comparable to existing state-of-the-art methods and that the reports generated were of notably good quality for humans reading them.

These systems show that by incorporating human feedback, sophisticated tasks can be solved. However, these systems are dependent on human feedback because LLMs to date are not capable of true reasoning. There are several limitations to this: the system may struggle with designing innovative experiments beyond standard methodologies, particularly in areas requiring creative problem-solving or novel approaches. The system still generates errors in the code (bugs or inefficiencies), it continues to maintain a high computational cost (several LLM calls), communication between agents is not yet perfect, report generation is still suboptimal in comparison to an expert, it generalizes poorly to highly specialized or niche research areas (they are poorly represented in training data and literature), and several ethical issues remain open.

In this section, we looked at different systems with a single agent or multiple agents. In the next section, we will see how HuggingGPT works in practice and how we can create multi-agent systems.

Working with HuggingGPT

There are two ways you can use HuggingGPT:

- Clone the repository locally
- Use the web service

Here, we will look at the two methods. The main difference is that when we clone the repository locally, we download all the models, and the system execution will be conducted locally. In contrast, the web service method requires that the execution is conducted in a service. In both cases, all models are used in inference; the difference lies in where the models are executed and the resources employed. Additionally, both approaches support the use of a web-based GUI.

Using HuggingGPT locally

To clone HuggingGPT (the corresponding repository is called Jarvis), it is useful to use Git LFS. Git LFS is an open source extension of Git. Git is designed to manage code repositories but not large binary files (such as videos, datasets, or high-resolution images). Git LFS is crucial for repositories that include large assets (e.g., datasets, videos, or binaries) because Git is otherwise inefficient at handling large files. Git LFS solves this problem by storing large files outside the regular repository objects and replacing them with lightweight references (pointers) in the Git repository. Git LFS keeps repository size manageable by storing large files outside the repository's regular objects, allows for better standardization when using large objects, and improves performance during operation with GitHub repositories (such as cloning, pushing, and pulling). The pointer contains various metadata about the file (e.g., size, hash, and location), and when we clone a repository, Git LFS downloads the files by exploiting the information in these pointers. This then allows us to separate operations on the code from those conducted on the large files. In general, it is common to use Git LFS for projects involving ML, game development, or video editing, because it allows for simplification and speeding up of the download

process. In ML projects, the model weights are very large and can be frequently updated; using Git LFS allows us to efficiently track and manage these files—such as downloaded models—without bloating the main repository without bloating the repository. As we mentioned, HuggingGPT uses several large models (for example, there are different diffusion models that can occupy several gigabytes), and Git LFS allows for easier management.

To install Git LFS, you can go to the official website (**https://git-lfs.github.com/**) and download the installer for your operating system (Windows, macOS, or Linux). Run the downloaded installer. On macOS, double-click the **`.pkg`** file or use the Homebrew package manager:

```
brew install git-lfs
```

Run the following command to enable Git LFS for your user:

```
git lfs install
```

Once you have installed Git LFS as a Git extension on your computer, it will automatically recognize and track when there are large files in the repository and manage them. It modifies or creates a few Git configuration entries (such as in **`~/.gitconfig`**) so that future clones and repositories you create can use LFS without extra hassle.

Cloning an LFS-enabled repository is as simple as if it were a regular repository (Git LFS takes care of the files in the background and large files are managed automatically):

```
git clone https://github.com/example/repo.git
```

If we want, we can easily conduct large file tracking:

```
git lfs track "*.bin"
git add .gitattributes
git commit -m "Track large .bin files with LFS"
```

Git LFS is compatible with classical Git commands. Pull/push operations are conducted as in normal Git workflows—no special steps are required unless a repository demands specific credentials or tokens.

At this point, we can proceed with the installation of HuggingGPT. The HuggingGPT repository is stored at **https://github.com/microsoft/JARVIS**.

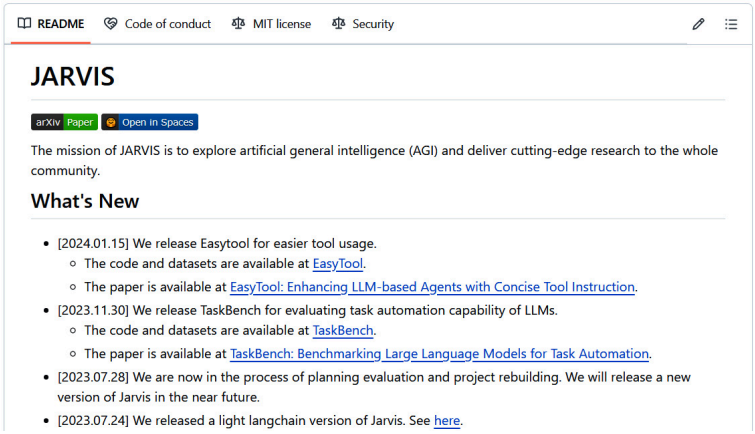


Figure 9.29 – Microsoft HuggingGPT

The first step is to clone the repository:

```
git clone https://github.com/example/microsoft/JARVIS.git
```

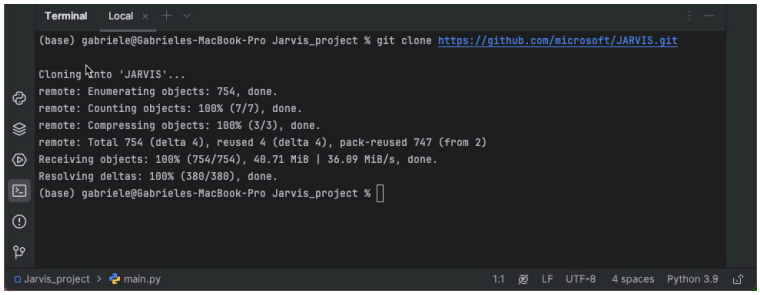


Figure 9.30 – Microsoft HuggingGPT cloning

The **`git clone`** command initiates the download of the repository from the remote URL. The terminal output indicates the repository being downloaded: objects (metadata and changes) and delta compression (a process that minimizes the amount of data transmitted by only sending differences between versions). Notice the following:

- **`Receiving objects: 100% (150/150), done.`**: This confirms that all objects (files and history) have been received
- **`Resolving deltas: 100% (85/85), done.`**: Git reconstructs the actual repository state by applying the changes (deltas) received

Once we have cloned the repository, we can go to the local repository (the local folder):

```
cd JARVIS/hugginggpt/server
```

This step is in preparation for creating or managing the **`conda`** environment, ensuring that the actions are performed in the context of the relevant project directory.

Then, we create a new **`conda`** environment named **`jarvis`** (or we can choose another name) and specify that it should use Python version 3.8:

```
conda create -n jarvis python=3.8
```

Note that **`-n`** means we want a new environment for our project, and **`python=3.8`** means we are explicitly defining the Python version to be 3.8 for this environment.

A **`conda`** environment allows us to isolate dependencies and avoid conflicts with global Python installations or other projects.

Note that **`conda`** is handling the following processes:

- **Collecting package metadata**: **`conda`** fetches information about the required packages and dependencies from its repositories. This ensures compatibility between Python 3.8 and any other libraries to be installed.
- **Solving the environment**: **`conda`** resolves potential dependency conflicts and finalizes the list of packages to be installed.

Since you may have installed **`conda`** previously, we just need to update it:

```
conda update -n base -c defaults conda
```

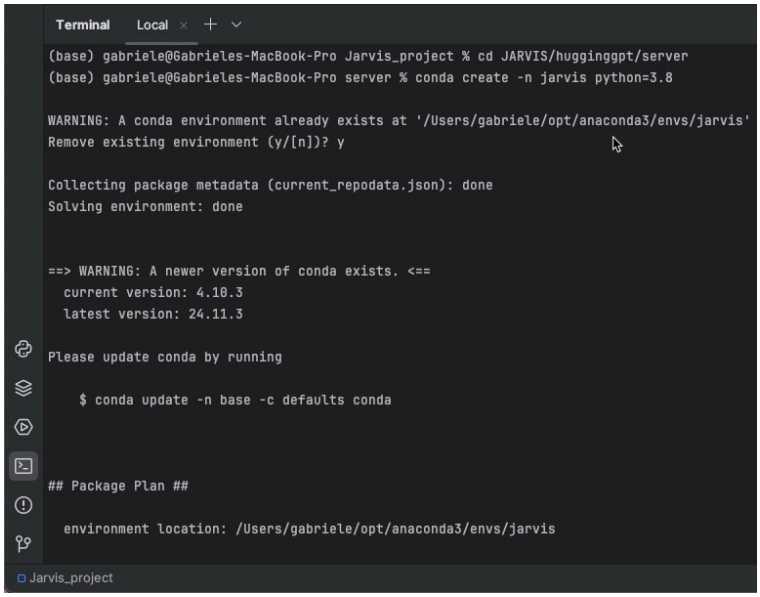


Figure 9.31 – Updating conda

After solving the environment and preparing to create it, **`conda`** installs the required base packages for the new environment. Each package is listed alongside the repository (**`pkgs/main`**) and its specific version (in this case, we are using macOS).

The terminal prompts us with **`Proceed ([y]/n)?`**. Remember to respond with **`y`** to confirm the installation of these packages.

Note these elements:

- **Preparing transaction**: **`conda`** ensures that the necessary dependencies are ready to be installed without conflicts
- **Verifying transaction**: It checks the integrity of the package metadata and ensures compatibility between all packages
- **Executing transaction**: **`conda`** installs the packages into the specified environment

Once these steps are completed, the new environment (**jarvis**) is ready for use.

Upon successful creation, **conda** provides the user with commands for managing the new environment.

To activate this environment, use the following:

```
conda activate jarvis
```

To deactivate an active environment, use the following:

```
conda deactivate
```

Remember that the activation switches the user's terminal session to use the **jarvis** environment, isolating its dependencies and Python version. Notice that the prompt changes from **(base)** to **(jarvis)**, indicating that the terminal is now operating within the **jarvis** environment. The environment's isolated Python version (3.8) and its dependencies are now being used. Any libraries or tools installed from this point will remain confined to this environment, avoiding interference with other projects.

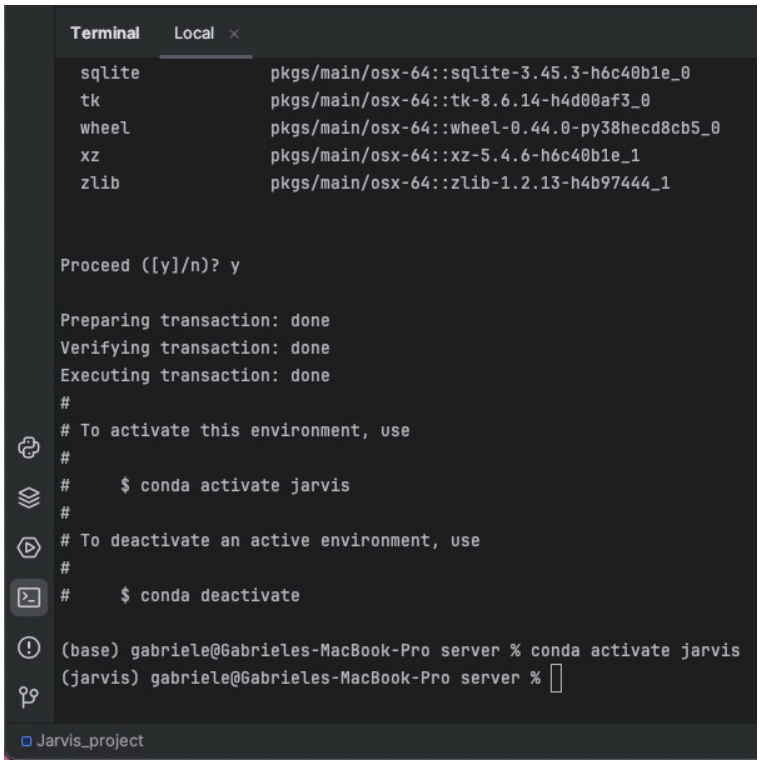


Figure 9.32 – conda activation

At this point, we begin to install the various requirements:

```
conda install pytorch torchvision torchaudio pytorch-cuda=11.7 -c pytorch -c nvidia
```

The following command uses **pip** to install dependencies listed in a **requirements.txt** file (most often, a list of packages is provided in a requirements file). These requirements are necessary to install HuggingGPT:

```
pip install -r requirements.txt
```

The following comment in HuggingGPT emphasizes that Git LFS must be installed. This script (provided as part of the project) automates the download of model files required for local or hybrid inference modes. As a reminder, local means the model runs entirely on the local machine and hybrid means the inference involves a mix of local and remote execution, as was described in the HuggingGPT paper (https://arxiv.org/abs/2303.17580) and in the preceding section:

```
# download models. Make sure that `git-lfs` is installed.
bash download.sh # required when `inference_mode` is `local` or `hybrid`
```

Once we have installed everything, we can start the execution:

```
python model_server.py --config config/config.default.yaml # required when `inferen
python awesome_chat.py --config config/config.default.yaml --mode server # for text
```

There are different scripts in the repository:

- **`model_server.py`**: This script runs a model server, which processes ML models based on the configuration file (**`config/config.default.yaml`**). The configuration file specifies parameters such as inference mode (local or hybrid), paths to the models, and hardware requirements.
- **`awesome_chat.py`**: This script starts a server for text generation or chatbot functionality.

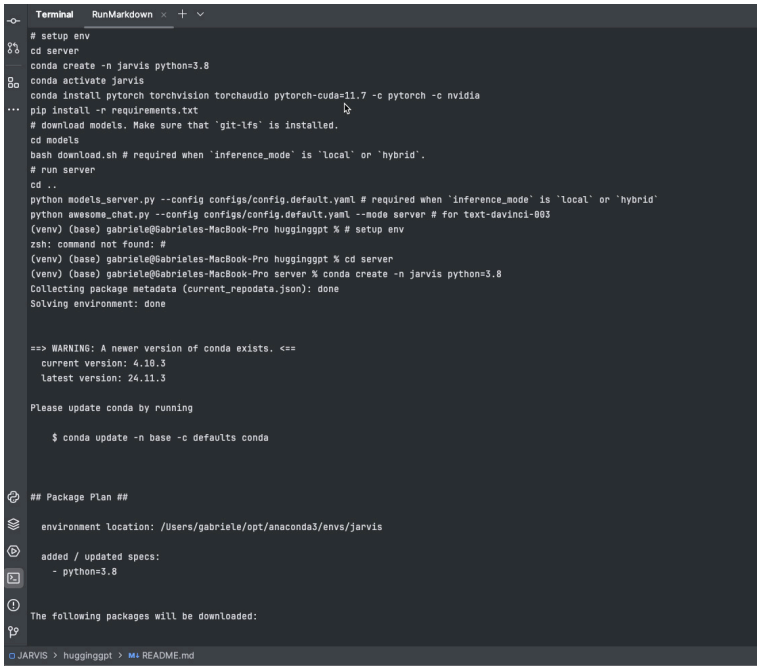


Figure 9.33 – Microsoft HuggingGPT finalizing installation

Since we have initialized **`awesome_chat.py`**, we can use a user-friendly web page.

Using HuggingGPT on the web

If you do not want to install HuggingGPT, you can use the online suite instead (on Hugging Face Gradio: **https://huggingface.co/gradio**). **Hugging Face Gradio** is a Python library that simplifies the process of creating user-friendly web-based interfaces for ML models and other Python applications.

With Gradio, developers can quickly build interactive demos for tasks such as text generation, image classification, and audio processing. These interfaces allow users to test models directly in their browser by providing inputs (e.g., text, images, or audio) and viewing real-time outputs. Gradio is highly customizable, supports integration with popular ML frameworks (such as PyTorch, TensorFlow, and Hugging Face models), and enables easy sharing of demos through public links or embedding in web applications.

The authors created a Gradio interface (launching Jarvis from local allows such an interface). The Gradio space can be accessed here: **https://huggingface.co/spaces/microsoft/HuggingGPT**.

As said, HuggingGPT is a system that connects LLMs with the ML community. As seen previously in the description of the system and its installation, the web interface also does exactly the same: connect an LLM with a set of ML models that are hosted on Hugging Face. In the web interface, only a few models are deployed on the **`local/inference`** endpoint due to hardware limitations (this interface serves as an example to understand and see in action how the system works).

Note that we need two tokens, which a user needs to obtain from each website:

- **Hugging Face token**: This is a personal authentication key that allows users to securely access Hugging Face's services, including their API, models, datasets, and other resources hosted on the platform. This token acts as an identifier for your account, ensuring that your requests to Hugging Face's systems are authorized and linked to your account. The token is then used to authenticate and use the models in inference. Hugging Face enforces rate limits for some services, especially for web inference.
- **OpenAI key**: This is a unique authentication key provided by OpenAI that enables developers to securely access and interact with OpenAI's APIs and services, such as GPT (e.g., GPT-3.5 or GPT-4), DALL·E, Codex, and Whisper. This key acts as a personalized credential that identifies your account and authorizes your usage of OpenAI's platform. The key is required to authenticate requests sent to OpenAI's API endpoints. OpenAI uses your API key to track your usage (e.g., the number of API calls made and tokens processed) and bill your account accordingly. In this case, the connection to GPT-4 is used.

Once we have our tokens ready, we can enter our question and click **Submit**.

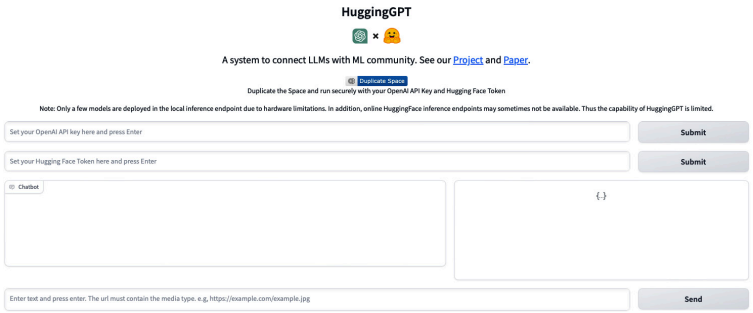


Figure 9.34 – HuggingGPT interface

We can see that there are two main panels:

- **Left panel**: A text input box labeled **Chatbot** is provided. This field is intended for user inputs, such as questions or commands, to interact with the HuggingGPT system.
- **Right panel**: There is an empty box next to the chatbot reserved for responses or outputs generated by HuggingGPT.

Below the chatbox, there is a button labeled **Send**, allowing users to submit their queries to HuggingGPT.

Note that the system already provides ready-made examples that we can use:

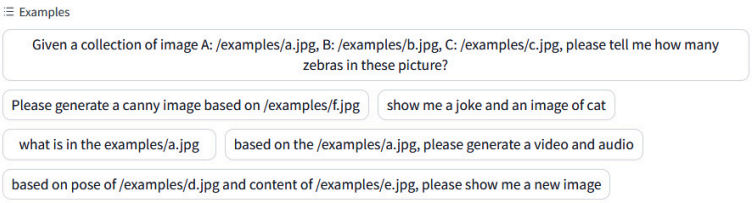


Figure 9.35 – HuggingGPT interface provided examples

We enter our tokens for both OpenAI and Hugging Face. Using the text input box labeled **Chatbot**, we can send natural language queries to HuggingGPT (“*Can you tell me which kind of pizza you see in the picture?*”) and send the query with the **Send** button. In addition, images or other multimedia elements can be added (in our case, we have added a picture of a pizza):

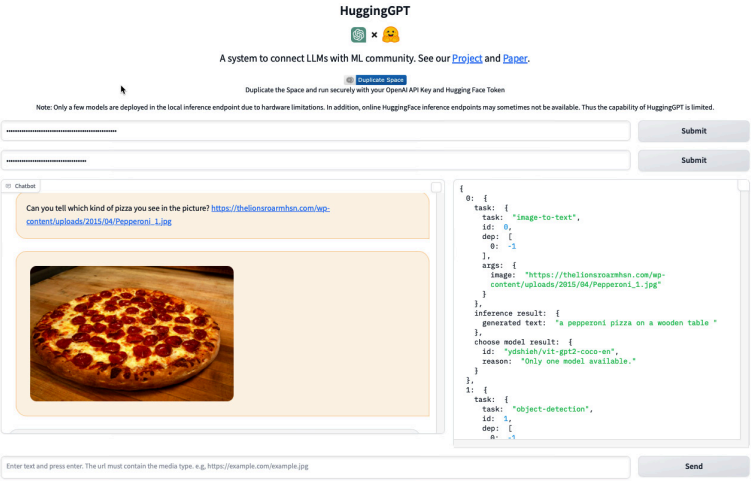


Figure 9.36 – Example of HuggingGPT interaction

In the panel on the right-hand side of the figure, we see the process that the system is working through: *1 pepperoni pizza on a wooden table.* This indicates that the system successfully processed the input image and

identified the object depicted as *pepperoni pizza*. This is a typical object detection task, and the system is using a model to identify the object (it is not an LLM that conducts the image recognition but a specialized model that is invoked by the LLM).

The chatbot provides a detailed answer based on the inference results:

```
Sure, based on the inference results, the pizza in the picture is a pepperoni pizza
```

HuggingGPT explains the process:

1. The first step involves the use of an image-to-text model to get a description of the image. **ViT-GPT2-COCO-EN** is a vision-language model that combines a **Vision Transformer (ViT)** for image encoding and **GPT-2** for natural language generation, fine-tuned on the **COCO dataset** for image captioning tasks. The model generates descriptive captions in English for input images, effectively translating visual content into coherent textual descriptions. It leverages the power of ViT for extracting detailed image features and GPT-2's language generation capabilities to produce accurate and contextually rich captions.
2. Then, HuggingGPT uses an object detection model to identify objects within an image. This object detection model also provides a similar response because it identifies both a pizza and a dining table. **DETR-ResNet-101** is a vision model designed for object detection and image segmentation. It combines a **ResNet-101** backbone (a convolutional neural network) for feature extraction with a **transformer-based architecture** for detecting and localizing objects in an image. **DEtection TRansformer (DETR)** uses transformers to model global relationships in an image, allowing for more accurate object detection without the need for traditional region proposal networks.
3. Then, a visual-answering model confirms what type of pizza is in the image. **ViLT-B/32-Finetuned-VQA** is a vision-and-language transformer model fine-tuned for **Visual Question-Answering (VQA)** tasks. It combines a lightweight **Vision-and-Language Transformer (ViLT)** architecture with a patch-based image tokenizer and transformer layers to process both visual and textual inputs jointly. The B/32 refers to the use of a 32 x 32 pixel patch size for image encoding. Fine-tuned specifically for VQA datasets, the model is designed to answer natural language questions about input images by reasoning over the visual and textual information.
4. Finally, the LLM observes that the three models are in agreement and thus is confident in responding.

To recap, HuggingGPT receives a request from the user and selects patterns. These patterns are executed, and outputs are collected. The system analyzes what these outputs are and generates a final response.

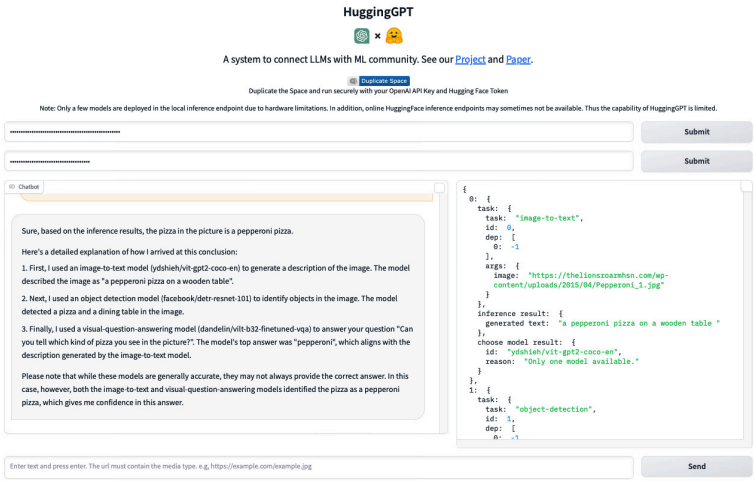


Figure 9.37 – Example of HuggingGPT response

HuggingGPT shows with a simple example how a multimodal task can be solved with an LLM. This is all done using information in the prompt and a set of tools.

In this section, we have seen a single LLM (a single agent) process a task, divide it into subtasks, and execute different models. A more elegant approach is to use multiple agents that approach a task from different perspectives, collaborate, and interact to solve a task. In the next subsection, we will see how this can be achieved.

Multi-agent system

In this section, we see how we can create a system that considers different agents and a set of tools (such as ML models). The entire code can be found in the **`Multi_Model-Travel_Planning_System.py`** script.

As a general overview, the system implements a travel planning assistant that uses several agents to create personalized travel plans. The system then combines weather prediction, hotel recommendations, itinerary planning, and email summarization. In other words, we have four different agents, each dealing with a different aspect of travel planning:

- **`WeatherAnalysisAgent`**: Uses a random forest regressor to predict the best time to visit a location based on historical weather data. Trains on past weather data (month, latitude, longitude, and weather score) and predicts the best months for travel based on weather scores. This agent then uses an ML model to conduct predictions (a model that is trained specifically for the system).
- **`HotelRecommenderAgent`**: Uses Sentence Transformer embeddings to find hotels based on user preferences. Stores hotel descriptions and converts them into embeddings, after which it matches user preferences with the most relevant hotels using semantic similarity. This agent, based on user preferences, searches its library for possible solutions.
- **`ItineraryPlannerAgent`**: Uses GPT-2 (text-generation pipeline) to create personalized travel itineraries. The agent generates trip plans based on destination, weather prediction, and hotel recommendations.
- **`SummaryAgent`**: Uses GPT-2 to generate a summary email for the client. This summary includes the hotel cost (per night cost × duration)

and additional daily expenses. After that, it generates a personalized email with trip details, cost breakdown, and itinerary highlights.

The following figure presents a schema of the agents and the process:

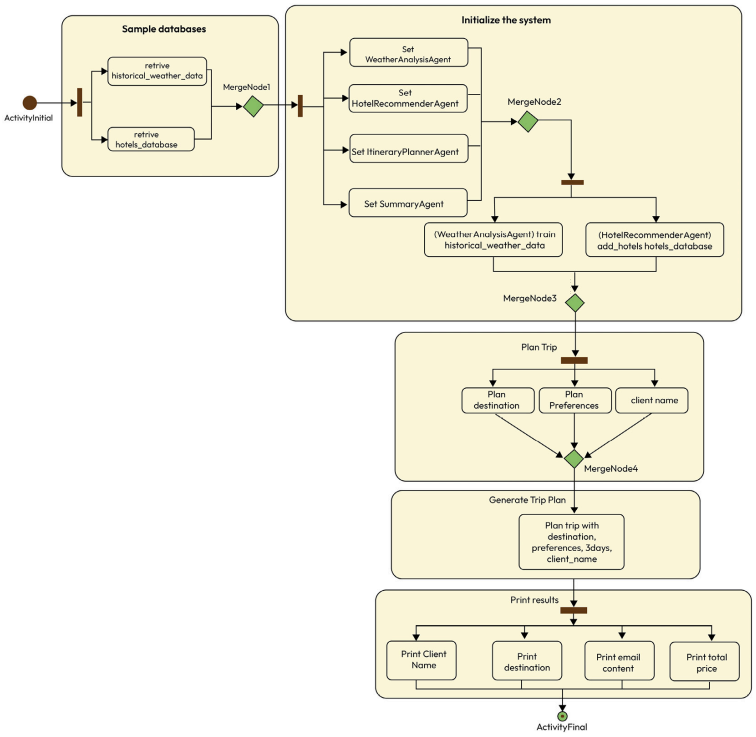


Figure 9.38 – Activity diagram of the AI Travel Planning System workflow showing the full sequence from data loading and agent initialization to trip planning and result output

`TravelPlanningSystem` links all agents together and is basically the main controller of the system. The system thus mimics this flow:

1. The user provides the destination, preferences, and duration.
2. The weather agent predicts the best time to visit.
3. The hotel agent finds matching accommodation.
4. The itinerary agent creates daily plans.
5. The summary agent generates an email and calculates costs.

Going into detail, we can see that agents here are defined as classes. **`WeatherAnalysisAgent`** is an ML-based component that analyzes historical weather data and predicts the best months to visit a given location. It does this using a Random Forest Regressor. We can see it as an agent using an ML model to perform a task. This snippet is initializing the agent:

```
class WeatherAnalysisAgent:
def __init__(self):
          self.model = RandomForestRegressor(n_estimators=100)
```

This agent creates a **`RandomForestRegressor`** model (**`n_estimators=100`** means the model consists of 100 decision trees) that must learn patterns from historical weather data, and then must predict weather scores for different months and locations:

```
def train(self, historical_data: Dict):
        X = np.array([[d['month'], d['latitude'], d['longitude']] for d in historic
```

```
        y = np.array([d['weather_score'] for d in historical_data])
        self.model.fit(X, y)
```

As mentioned before, this model is not trained (i.e., it is not used in inference) but is trained on the spot. For this, we have within our class a **`train`** method. Random forest uses month, latitude, and longitude for a location to learn to predict a **`weather_score`** value (a numerical score representing how good the weather is in that month). In this snippet, the data is processed and the model is trained.

At this point, we can use **`predict_best_time`** as a method that predicts the best months to visit a location based on the trained weather model. In this case, the method takes only two inputs (the latitude and longitude of the location) and returns its predictions:

```
def predict_best_time(self, location: Dict) -> Dict:
        # Predicts the best time to visit a location based on weather patterns
        predictions = []
        for month in range(1, 13):
            # predict returns a 2D array, we take the first (and only) element
            prediction = self.model.predict([[
                month,
                location['latitude'],
                location['longitude']
            ]]).item()  # .item() converts numpy array to scalar
            predictions.append({'month': month, 'score': float(prediction)})
        return {
            'best_months': sorted(predictions, key=lambda x: x['score'], reverse=Tr
            'location': location
        }
```

Note that we initialize predictions, which will contain all scores for 12 months (in fact, predictions are conducted in a loop through all 12 months, from January to December). Finally, we reorder the list from best to worst to identify the best months to visit. The method then returns a list of the top three months with the highest predicted weather scores.

`HotelRecommenderAgent` is a hotel recommendation system that utilizes semantic similarity to match hotels with user preferences and uses natural language processing (**NLP**) to understand and compare hotel descriptions and user preferences:

```
class HotelRecommenderAgent:
    def __init__(self):
        self.encoder = SentenceTransformer('all-MiniLM-L6-v2')
        self.hotels_db = []
        self.hotels_embeddings = None
```

During agent initialization, **`all-MiniLM-L6-v2`** (a pre-trained NLP model designed for semantic similarity) is loaded. This model is an embedder (as described in ***Chapter 5***), converting text (hotel descriptions and user preferences) into vector embeddings (numerical representations in a multi-dimensional space). Once we have vectors, we can measure the similarity between two vectors (user preferences and hotel descriptions). The agent retrieves the available hotels (**`self.hotels_db`**) and can store precomputed embeddings (numerical vectors) for all hotel descriptions (**`self.hotels_embeddings`**).

Next, in the following snippet, we have **`add_hotels`**, which adds hotels to the database and computes the embedding for the description, and then adds it to our embeddings database. **`find_hotels`** finds hotels that match the user's preferences using semantic similarity:

```
def add_hotels(self, hotels: List[Dict]):
        self.hotels_db = hotels
        descriptions = [h['description'] for h in hotels]
        self.hotels_embeddings = self.encoder.encode(descriptions)
    def find_hotels(self, preferences: str, top_k: int = 5) -> List[Dict]:
        pref_embedding = self.encoder.encode([preferences])
        similarities = np.dot(self.hotels_embeddings, pref_embedding.T).flatten()
        top_indices = similarities.argsort()[-top_k:][::-1]
        return [
            {**self.hotels_db[i], 'similarity_score': float(similarities[i])}
            for i in top_indices
        ]
```

What happens is that we conduct embedding of a user's preferences and then compute the cosine similarity with all stored hotel vectors. In this case, we then select the five hotels that are closest to our hotel description (**`top_k=5`** means selecting the top five hotels).

`ItineraryPlannerAgent` is responsible for automatically generating travel itineraries based on destination information (city or attractions), weather predictions (best months to visit), hotel recommendations (selected accommodation), and trip duration (number of days). It uses a natural language model (GPT-2) to generate customized travel itineraries based on these inputs:

```
class ItineraryPlannerAgent:
    def __init__(self):
        # Uses a language model for generating itineraries
        self.planner = pipeline(
            "text-generation",
            model="gpt2",  # In production, use a more powerful model
            max_length=500,
            truncation=True,
            pad_token_id=50256
        )
```

The agent initializes an NLP model (GPT-2 Model, which is a pre-trained language model for text generation) using the Hugging Face transformers library. We select a pipeline that is focused on text generation (**`"text-generation"`** means the model will generate text based on a prompt). Other parameters mean we limit the generated text to 500 tokens (**`max_length=500`**) and we ensure truncation.

Since we interact with LLMs through prompts, we have a method that allows us to create a structured prompt that we will then use to interact with the model. This prompt is designed to be able to generate a travel plan, where it enters some specific information: the length of stay (duration), the destination, weather information (the best mounts we identified earlier), hotel selection (which were identified with the previous agent), and a list of attractions:

```
def _create_prompt(self, destination_info: Dict, weather_info: Dict,
                   hotel_info: Dict, duration: int) -> str:
    return f"""Create a {duration}-day itinerary for {destination_info['name']}.
Weather: {weather_info['best_months'][0]['month']} is the best month.
Hotel: Staying at {hotel_info[0]['name']}.
Attractions: {', '.join(destination_info['attractions'])}."""
```

At this point, we can create the itinerary; the **`create_itinerary`** method precisely takes the previous prompt that contains all the information we need (destination, weather, hotel selection, and trip duration). Inside the **`create_itinerary`** method is a method called **`_create_prompt`** to generate the prompt. The GPT-2 model takes the input prompt and produces a detailed itinerary:

```
def create_itinerary(self, destination_info: Dict, weather_info: Dict,
                     hotel_info: Dict, duration: int) -> Dict:
    prompt = self._create_prompt(destination_info, weather_info, hotel_info, durati
    #Generate the itinerary
    response = self.planner(prompt)[0]['generated_text']
    return {
        'itinerary': response,
        'duration': duration,
        'destination': destination_info['name']
    }
```

The final agent, that is, **`SummaryAgent`**, is responsible for summarizing trip details, calculating the total estimated cost, and generating a personalized email for the client using GPT-2. Our agent is initialized similar to the previous agent; the only difference is that in this case, the generation length is greater (**`max_length=1000`**):

```
class SummaryAgent:
    def __init__(self):
        # In production, use a more powerful LLM like GPT-4 or Claude
        self.llm = pipeline(
            "text-generation",
            model="gpt2",
            max_length=1000,
            truncation=True,
            pad_token_id=50256
        )
```

`calculate_total_price` is a tool that is used by the agent to be able to calculate the total cost of the trip (remember that LLMs are not good at arithmetic, so it is better to use an external tool):

```
def calculate_total_price(self, hotel_info: Dict, duration: int) -> float:
        # Calculate total trip price
        hotel_cost = hotel_info[0]['price'] * duration
        # Estimate additional costs (activities, meals, transport)
        daily_expenses = 100  # Simplified example
        additional_costs = daily_expenses * duration
        return hotel_cost + additional_costs
```

The agent does a series of very simple calculations:

- The hotel price per night is multiplied by the duration of the stay

- A fixed daily expense of $100 is used to estimate costs for meals, transport, activities, and sightseeing tickets
- Hotel and additional costs are added to return the final estimate

`create_email` allows you to create the email summary that will be sent to the customer:

```
def create_email(self, trip_data: Dict, client_name: str) -> Dict:
        total_price = self.calculate_total_price(
            trip_data['recommended_hotels'],
            trip_data['itinerary']['duration']
        )
        prompt = f"""
        Dear {client_name},
        Based on your preferences, I'm pleased to present your travel plan:
        Destination: {trip_data['itinerary']['destination']}
        Duration: {trip_data['itinerary']['duration']} days
        Best time to visit: Month {trip_data['weather_analysis']['best_months'][0][
        Recommended Hotel: {trip_data['recommended_hotels'][0]['name']}
        Itinerary Overview:
        {trip_data['itinerary']['itinerary']}
        Estimated Total Cost: ${total_price}
        Please let me know if you would like any adjustments.
        """
        # Generate email using LLM
        response = self.llm(prompt)[0]['generated_text']
        return {
            'email_content': response,
            'total_price': total_price,
            'summary_data': {
                'destination': trip_data['itinerary']['destination'],
                'duration': trip_data['itinerary']['duration'],
                'hotel': trip_data['recommended_hotels'][0]['name'],
                'best_month': trip_data['weather_analysis']['best_months'][0]['mont
            }
        }
```

As we can see, the email will be structured to include costs (we use the method described previously) and the other information we obtained earlier. Note that we use a template.

Remember, **`TravelPlanningSystem`** is the main controller that integrates all AI agents for automated travel planning:

```
class TravelPlanningSystem:
    def __init__(self):
        self.weather_agent = WeatherAnalysisAgent()
        self.hotel_agent = HotelRecommenderAgent()
        self.itinerary_agent = ItineraryPlannerAgent()
        self.summary_agent = SummaryAgent()
```

In the first step, we initialize our four agents. Each agent will handle a specific task. If you noticed, we have used a modular system. The advantages of this are as follows:

- Each component operates independently, making the system scalable
- Components can be updated or replaced without affecting others
- It follows the **Single Responsibility Principle** (**SRP**) for clean code architecture

At this point, we can start the setup – getting the best hotels and the best months to visit:

```
def setup(self, historical_weather_data: Dict, hotels_database: List[Dict]):
        # Initialize and train the models
        self.weather_agent.train(historical_weather_data)
        self.hotel_agent.add_hotels(hotels_database)
```

Finally, you have to coordinate the entire trip and then generate the summary email with cost estimates and the itinerary:

```
def plan_trip(self, destination: Dict, preferences: str, duration: int, client_name
        # 1. Weather analysis and best time prediction
        weather_analysis = self.weather_agent.predict_best_time(destination)
        # 2. Hotel search
        recommended_hotels = self.hotel_agent.find_hotels(preferences)
        # 3. Itinerary creation
        itinerary = self.itinerary_agent.create_itinerary(
            destination,
            weather_analysis,
            recommended_hotels,
            duration
        )
        # 4. Create summary email and calculate price
        trip_data = {
            'weather_analysis': weather_analysis,
            'recommended_hotels': recommended_hotels,
            'itinerary': itinerary
        }
        summary = self.summary_agent.create_email(trip_data, client_name)
        return {
            **trip_data,
            'summary': summary
        }
```

Now that we have created the multi-agent platform, we have to execute it. The **`main()`** function serves as the entry point for running the *Travel Planning System*. It demonstrates the system's functionality by doing the following:

1. Initializing sample data (weather history and hotels)
2. Setting up and training AI models
3. Executing the travel planning process
4. Printing the generated trip summary

We provide the system with various information about the weather, destination, hotels, and so on. After that, the system is initialized and executed. At this point, it prints travel summary details and the personalized email generated by GPT-2, and it shows the estimated total trip cost:

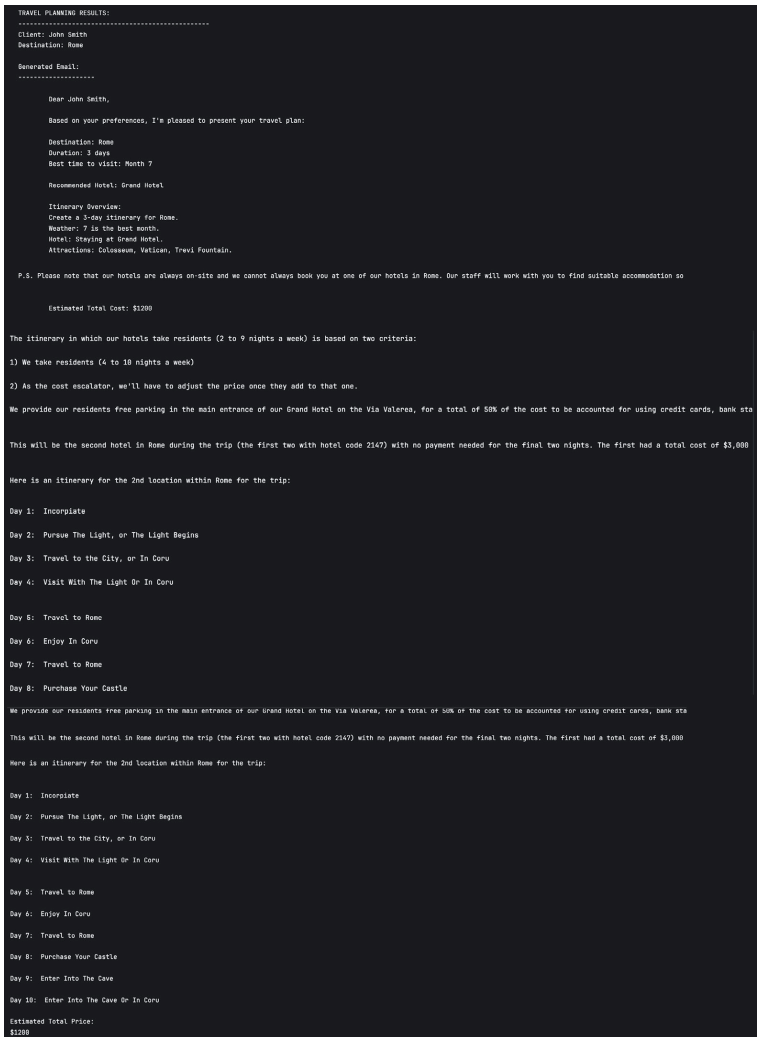
```
def main():
    # Example data with a full year of weather information
    historical_weather_data = […    ]
    # Sample hotel database
    hotels_database = […]
    # Initialize the system
    system = TravelPlanningSystem()
    system.setup(historical_weather_data, hotels_database)
    # Plan a trip
```

```
    destination = {
        'name': 'Rome',
        'latitude': 41.9028,
        'longitude': 12.4964,
        'attractions': ['Colosseum', 'Vatican', 'Trevi Fountain']
    }
    preferences = """Looking for a luxury hotel in the city center,
    preferably with spa facilities and fine dining options"""
    client_name = "John Smith"
    # Generate trip plan
    trip_plan = system.plan_trip(destination, preferences, duration=3, client_name=
    # Print results in a readable format
    print("\nTRAVEL PLANNING RESULTS:")
    print("-" * 50)
    print(f"Client: {client_name}")
    print(f"Destination: {destination['name']}")
    print("\nGenerated Email:")
    print("-" * 20)
    print(trip_plan['summary']['email_content'])
    print("\nEstimated Total Price:")
    print(f"${trip_plan['summary']['total_price']}")
```

Ensure that the **`main()`** script runs only if the script is executed directly:

```
if __name__ == "__main__":
    main()
```

At this point, we just have to test it. Once you have run the script, this should be the result:



```
TRAVEL PLANNING RESULTS:
--------------------------------------------------
Client: John Smith
Destination: Rome

Generated Email:
------------------

        Dear John Smith,

        Based on your preferences, I'm pleased to present your travel plan:

        Destination: Rome
        Duration: 3 days
        Best time to visit: Month 7

        Recommended Hotel: Grand Hotel

        Itinerary Overview:
        Create a 3-day itinerary for Rome.
        Weather: 7 is the best month.
        Hotel: Staying at Grand Hotel.
        Attractions: Colosseum, Vatican, Trevi Fountain.

 P.S. Please note that our hotels are always on-site and we cannot always book you at one of our hotels in Rome. Our staff will work with you to find suitable accommodation so


        Estimated Total Cost: $1200

The itinerary in which our hotels take residents (2 to 9 nights a week) is based on two criteria:

1) We take residents (4 to 10 nights a week)

2) As the cost escalator, we'll have to adjust the price once they add to that one.

We provide our residents free parking in the main entrance of our Grand Hotel on the Via Valerea, for a total of 50% of the cost to be accounted for using credit cards, bank sta


This will be the second hotel in Rome during the trip (the first two with hotel code 2147) with no payment needed for the final two nights. The first had a total cost of $3,000


Here is an itinerary for the 2nd location within Rome for the trip:

Day 1:  Incorpiate

Day 2:  Pursue The Light, or The Light Begins

Day 3:  Travel to the City, or In Coru

Day 4:  Visit With The Light Or In Coru


Day 5:  Travel to Rome

Day 6:  Enjoy In Coru

Day 7:  Travel to Rome

Day 8:  Purchase Your Castle

We provide our residents free parking in the main entrance of our Grand Hotel on the Via Valerea, for a total of 50% of the cost to be accounted for using credit cards, bank sta

This will be the second hotel in Rome during the trip (the first two with hotel code 2147) with no payment needed for the final two nights. The first had a total cost of $3,000

Here is an itinerary for the 2nd location within Rome for the trip:

Day 1:  Incorpiate

Day 2:  Pursue The Light, or The Light Begins

Day 3:  Travel to the City, or In Coru

Day 4:  Visit With The Light Or In Coru


Day 5:  Travel to Rome

Day 6:  Enjoy In Coru

Day 7:  Travel to Rome

Day 8:  Purchase Your Castle

Day 9:  Enter Into The Cave

Day 10:  Enter Into The Cave Or In Coru

Estimated Total Price:
$1200
```

Figure 9.39 – Screenshots showing the execution

This *Travel Planning System* is a prototype demonstrating how AI agents can collaborate to automate a real-world problem.

Of course, a whole series of improvements can be made to make the system more useful:

- The data used is static (it is a toy example). You could connect with a number of APIs to obtain real-time data for the weather (OpenWeatherMap or AccuWeather), hotels (Booking.com or Expedia API), and destinations (Google Places API or Yelp). Extensions such as flights and transportation could also be added (Google Flights API or Rome2Rio).
- GPT-2 is outdated (we used it because it is much smaller than other models) and not fine-tuned for travel. You can replace GPT-2 with a larger or travel-optimized model. For example, you could use larger models such as GPT-4 or Claude, or open source alternatives such as LLaMA. Also, open source models can be fine-tuned on real travel itineraries from Tripadvisor, Lonely Planet, or Reddit.
- The itinerary is generic and not adaptable to different types of travelers. You could ask for different information from the traveler, such as budget preferences, what kinds of activities they prefer (cultural, adventure, food, family-friendly, and so on), or whether they need special accommodations (wheelchair, traveling with elderly, or pet-friendly). This requires a larger model, and you can also test recommendation models. In addition, there are methods and models that implement **Multi-Criteria Decision-Making** (**MCDM**) to conduct more sophisticated rankings.

In any case, this system, though simple, allows us to see several interesting elements:

- Instead of using one large monolithic AI model, the system is broken down into specialized agents. This idea can come in handy for modern software design.
- This simple example mimics how multi-agent AI platforms work in autonomous vehicles, finance, healthcare, and robotics. In fact, multi-agent collaboration is a system designed with scalability, modularity, and efficiency in mind, which are necessary for real-world applications.
- The system can dynamically generate personalized recommendations (although in our case, it is hardcoded, we are mimicking what happens when a user enters their preferences).
- The system also analyzes multiple factors (weather, hotels, and attractions) and optimizes travel plans. Modern systems that do something similar use precise ML models (we used random forest in our example), have vast databases (in our case, we are mimicking a database of hotels), take user preferences into account, and use automated systems to respond to the customer (our email).

Although this is a very simple system, we can think about how a similar system could be used in various other industries:

- AI medical assistants that recommend treatments, optimize hospital schedules, and predict disease risks
- AI shopping assistants that recommend products based on user preferences and purchase history

- Multi-agent AI systems for self-driving cars (navigation, pedestrian detection, or traffic optimization)
- AI-driven advisors that help with investment strategies, risk management, and fraud detection
- An AI-powered urban planner that optimizes traffic, energy use, and public transport routes

In this section, we looked at how to create a multi-agent system. In the next section, we will discuss how multi-agent systems fit into the various business models that exist today or are under greater development. This will provide an important perspective, as it will allow you to understand how to adapt your multi-agent platform to the needs of businesses.

SaaS, MaaS, DaaS, and RaaS

In this section, we will explore various business models influenced by recent advancements in AI. While multi-agent LLMs represent cutting-edge technology, their value lies in being adaptable to meet business needs, enabling them to be effectively packaged, marketed, and delivered to businesses and consumers. Considering that these systems are extremely expensive to develop and maintain, it is important for the reader to understand what the revenue models are so that they can think about, design, and develop products that align with the company's strategy. Understanding these models allows us to grasp that a multi-agent system is not a standalone item but should be considered a product and that this product can be marketed in various ways. In addition, LLMs are extremely expensive products, and each of these business models has advantages and disadvantages in terms of continuous updates, scalability, and flexibility in AI deployment. At the same time, these business models regulate access to technology whether you are interested in developing AI models or are a customer. These choices (about the platform, business models, and so on) must be made before the product is developed, and they determine its development, since the costs do not allow for trial and error. The choice of business model is defined by the structure of the product and the multi-agent system, as well as the economic viability of the company.

Software as a Service (SaaS)

SaaS is a service model in which software is hosted in the cloud by a provider and is made available to users over the internet. In the traditional model, software is provided to the user to be installed and used locally (on the user's device). SaaS, on the other hand, allows access over the internet, usually on the web browser or with a mobile app. Often, SaaS is provided via subscription rather than through a one-time purchase. The SaaS paradigm began in 1999 when Salesforce launched its **Customer Relationship Management** (**CRM**) as a cloud-hosted service. SaaS is now the most widely used sales paradigm by different companies, especially for **Business-to-Business** (**B2B**) applications. Its popularity is growing, and it is expected that SaaS software revenue will grow more and more in the coming years.

SaaS applications are typically built to be hosted in the cloud (they are called cloud-native). The company developing these apps can decide whether to host on its own infrastructure or leverage that of cloud service providers (examples are Google Cloud, IBM Cloud, OVH, Aruba, **Amazon**

Web Services (**AWS**), and Microsoft Azure). Given the demand for app providers, some providers create focused infrastructure for hosting these apps, and so we also talk about **Platform as a Service** (**PaaS**).

In PaaS solutions, a provider conducts hosting of both hardware and software through dedicated infrastructure that is made available to product developers. This allows developers to focus on coding without having to worry about maintaining or managing the infrastructure behind it. The platform allows the hosting of both the application and the data, or even the training of a model, leaving only the coding to the developer. This has enabled accelerated product development by many businesses, who have managed to avoid investing in expensive infrastructure (although extensive use of these platforms can have a high cost, especially when the applications are generative AI). Although PaaS allows a simplification of the process, developers are forced to conform their applications to the requirements of the platforms and environment. This is not always possible, resulting in difficulties in deployment or other issues. Therefore, an alternative paradigm has emerged that allows the user greater flexibility, control, and adaptability, especially when the application or business requires it. This paradigm is called **Infrastructure as a Service** (**IaaS**) and emerged around 2010. In IaaS, a user can access computing resources through web services, thus being able to rent infrastructure (servers, networking, and storage) as needed. The user retains more control over the infrastructure, while the provider focuses on the hardware (examples include Google Compute Engine, DigitalOcean, and Amazon Elastic Compute Cloud). PaaS and IaaS can thus be seen as extensions of SaaS or as services for businesses that need a supporting ecosystem.

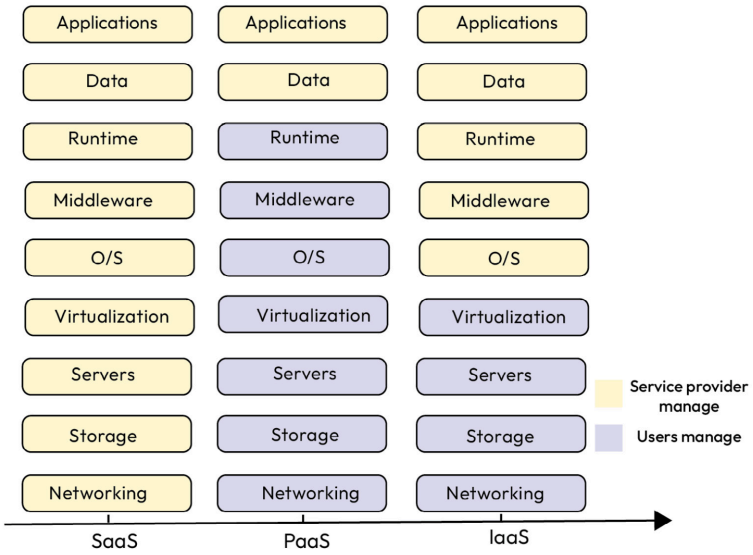


Figure 9.40 – Comparison between different paradigms (https://arxiv.org/pdf/2311.05804)

SaaS applications are therefore designed to be accessible via an internet connection from a device that must be connected to the internet in order to access the application (a device that is not connected cannot access the application and it is not a requirement to allow access locally). Software is developed to be used through a web browser or with a specific app (mobile software). Some SaaS applications (as in the case of Adobe Acrobat) may require the user to download and install a dedicated client (a light program, which is not the full application, that has to be installed on a local PC) on their computers (but this is generally a minority of cases). A SaaS application is generally a **multi-tenant software architec-**

ture, where a single instance of a software application (along with its database and hardware) serves different user accounts (or multiple tenants). A tenant is what is called a user of the software, and it is a user or group of users within an organization.

In SaaS, it is crucial to have an architecture that ensures each tenant's data is isolated and inaccessible to other tenants. This approach offers the advantage of cost reduction by enabling the software to be optimized for a single piece of hardware and infrastructure, which is then shared among all users. It also allows for greater scalability, easier customization, and maintenance (providers can conduct the update on their own infrastructure and on a single architecture).

SaaS is therefore one of the most widely used paradigms because it has a number of advantages:

- **Cost efficiency**: There are no upfront costs to the customer, such as expenses for hardware or a software license. In SaaS, the customer either pays by subscription or on a pay-as-you-go basis.
- **Scalability**: SaaS scales easily for the customer and does not require additional hardware. Similarly, software is structured to make it easy to scale up customers. In the case of AI models, the customer does not need large hardware but can directly leverage that provided by the provider.
- **Accessibility**: The customer can access the application from anywhere in the world via an internet connection. Also, using the web browser, the software is optimized for whatever hardware the client has. SaaS also reduces the barrier of access to AI for clients (fewer resources and less need for expertise) through the use of templates, APIs, and frameworks.
- **Ease of integration and customization**: It is much easier for the developer to provide updates, security patches, and maintenance, in terms of both resources and time. The ability to manage customization for the client is usually provided in an easier way, while at the same time maintaining control. Equally, for an AI system, updated templates can be provided.
- **Fast deployment**: SaaS reduces deployment and market access time by being immediately available in the marketplace.
- **Data and model sharing**: Model and data access can be easily allowed to users from different teams or in various locations simultaneously and effectively and efficiently.

There are, of course, also some limitations and disadvantages to SaaS:

- **Dependency on internet connectivity**: SaaS requires a stable connection, and connection disruptions can stop critical processes and errors. Rural areas and countries with little infrastructure may not be covered.
- **Limited customization**: SaaS solutions are developed with the idea of covering as much business as possible with one product. Typically, they provide a limited number of customization possibilities that may not cover all the needs of a particular business. This is also true in the case of an AI system; the client has little control over the models and the models may not be able to meet client requirements.
- **Data security and privacy concerns**: Hosting on third-party servers brings the risk of data breaches or unauthorized access. In addition,

there may be compliance issues with regulations in countries such as the European Union (e.g., data must be maintained on servers in certain countries). Training or using AI models may require having to share sensitive data, and this may be against GDPR or other regulations (as well as an additional privacy risk).

- **Vendor lock-in**: Businesses may remain anchored to a particular SaaS provider and then be unable to migrate to other platforms due to cost and complexity. In addition, different providers may terminate the service (or be acquired), increase costs abruptly, or eliminate features considered essential. SaaS can become expensive over a period of time, especially when subscription-based (some providers charge more as users increase).
- **Performance issues**: Shared resources in multi-tenant architectures can lead to slower performance during peak usage. In addition, there may be unexpected server downtime or maintenance schedules that hurt the business (for example, if maintenance is conducted at night on Pacific Time, it disrupts business hours in Europe) and over which the customer has no control. AI systems that must run in real time may have latency or performance problems (both in training and inference). In addition, the provider may not provide cutting-edge AI or may not have implemented it yet (or they may use models that do not fit the customer's needs).
- **High computational costs**: SaaS has an infrastructure cost for the developer, and in the case of AI, this cost can be higher (use of GPUs or large storage costs). Some of these services are particularly expensive for users.

Model as a Service (MaaS)

MaaS is a new paradigm that was born with the development of big data, AI, and Web 3.0. MaaS is a cloud computing-based service paradigm that offers AI and ML models and related IaaS to developers and enterprises.

MaaS seeks to simplify access to AI for businesses that have neither the expertise nor the infrastructure to train generative AI or broad models in general. MaaS enables the use of pre-trained ML models and algorithms through the use of simple interfaces, APIs, or the browser. Just like with SaaS, access to models is through the internet (and requires the business to have an internet connection). The provider must then conduct the hosting of the models and allow developers access to the models that have been trained. Developers can then use these models to add AI functions to their systems and apps. MaaS is often a platform where models that have been trained on a large amount of data or optimized for a possible task are hosted. MaaS reduces the complexity of managing these models (especially training and deployment) and allows developers to focus on using the models or how to integrate them for specific applications. Developers save time and resources since they do not have to train these models from scratch. MaaS thus has certain similarities to PaaS and IaaS but conducts an additional level of abstraction and focuses on AI solutions. In a sense, MaaS can be viewed as an intermediate solution between SaaS and PaaS or IaaS. It not only provides a service but also offers an infrastructure that enables the development of custom products.

Another difference between SaaS and MaaS is in the underlying architecture of the two paradigms. SaaS focuses on applications (application layer) that depend on an operating system (whether mobile or desktop appli-

cation) that allows them to run, as well as on a layer that allows the app to be hosted. In the case of MaaS, the architecture focuses on the model that needs a specific framework to be hosted.

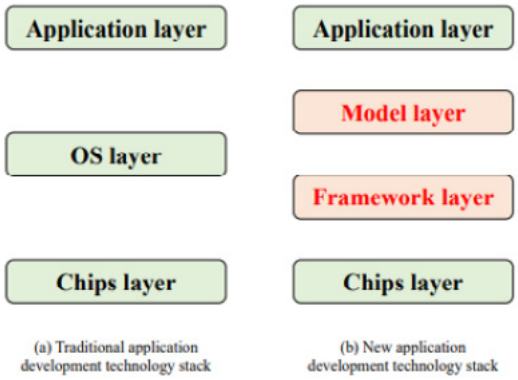


Figure 9.41 – Comparison between traditional and model-based technology stacks (https://arxiv.org/pdf/2311.05804)

In MaaS, the following elements are often present:

- **Cloud computing**: MaaS is based on an infrastructure on the cloud where various models are maintained and deployed. This allows easy access to the models and enables greater scalability.
- **Model training and optimization**: MaaS providers take care of the training of large models on large datasets. MaaS providers also take care of the entire ecosystem to enable more effective exploitation of models. For example, they can provide models of different sizes, including quantized or fine-tuned versions for specific applications.
- **API and development tools**: MaaS providers also provide APIs and tools that allow the developer to use the models for their applications easily. The purpose is to allow easy integration of models into other applications and infrastructures. So, the API acts as an endpoint, takes data, and returns predictions.
- **Monitoring and analytics**: To date, there is increasing focus on how to monitor models once they are in production. MaaS providers typically provide a number of tools to monitor model performance, identify the presence of issues, integrate feedback, or improve resource allocation.
- **Scalability, security, and privacy**: MaaS providers focus on the scalability of their systems by allowing customers to be able to manage multiple users at the same time (thus allocating different bandwidth, computing power, or storage as needed). At the same time, today there is more attention to privacy and security (especially as there is much more regulation). Platforms often have a number of tools to be able to increase the privacy and security of applications that integrate their models.

Hugging Face is an example of a MaaS provider. Hugging Face provides access to thousands of pre-trained models (from the company itself, other companies, or users) for computer vision, NLP, audio, video, and more. These models are hosted on their Model Hub and can be either used via an API or installed locally. So, a user who doesn't want to download models can use an inference API without owning the infrastructure needed to manage the model (this API uses the pay-as-you-go system). Developers who do not have the expertise or resources can directly use the endpoint API to directly integrate AI models within their applications. In addition, Hugging Face also offers a platform for hosting and deploying both the

model and application, extending MaaS capabilities and providing flexibility to customers who want to use their custom models. Hugging Face also provides tools to improve the scalability of models and open source libraries to facilitate model development or integration (e.g., Transformers, Datasets, Diffusers, sentence embedding, and so on), as well as offering a forum to enable user exchange, educational resources for users, and other services. There are other MaaS providers, such as Google AI (pre-trained models for NLP (Natural Language API), vision (Vision API), speech to text, translation, or custom model training with Vertex AI) and AWS (which offers pre-trained models for language, image, and text (e.g., AWS Comprehend, Rekognition, and Translate) or infrastructure for custom models).

MaaS has the following advantages, especially regarding the AI domain:

- **Simplified model development and deployment**: MaaS lowers the technical barrier to using generative AI. Companies do not need developers who are experts in the technology or different algorithms because most models are delivered via endpoints. This allows companies to focus on applications and model integration for their products. If needed, MaaS also simplifies the approach to fine-tuning models for their applications. MaaS, as opposed to SaaS, is tailored to the entire AI workflow and offers tools for deploying, training, managing, and scaling models, thus enabling better support for companies interested in using AI.
- **High performance and scalability**: The use of cloud computing facilitates system scaling. In fact, the use of AI can require high costs and large resources (especially when it comes to using LLMs), and MaaS allows for better resource management by facilitating access to large models without initial entry costs for different businesses. Typically, users pay for their consumption and receive computing according to their needs, thus enabling better performance and scalability. Since MaaS is optimized for AI workloads, it can scale easily when there are fluctuating computational demands (SaaS typically focuses on allocating a variable number of users, but users may have a different need for computing depending on the different usage of models).
- **Shared knowledge and collaboration**: MaaS is built on collecting large datasets and training large models. These pre-trained models can then be fine-tuned by developers interested in adapting the models to particular applications. This means that developers need to collect much less data and do not have to train large models from scratch. This saves both resources and costs (fine-tuning is much less computationally expensive than pre-training). In addition, MaaS allows standardization that reduces the technical knowledge required to be able to use these models and allows information and tutorials to be obtained easily. Models can then also be shared by the community on platforms on which both information and experiences are also exchanged (this promotes a collaborative environment and accelerates the development of new models).
- **Business support**: MaaS uses a flexible payment model, such as subscription based, where you pay only for current consumption. Generally, this solution is cost effective and affordable for many small businesses. It is convenient for providers because once they choose a technology and integrate it into their products, users remain loyal. Model integration allows businesses to gain insights in an easy and in-

expensive way (models for forecasts or other predictions, report writing, and visualizations).

- **Flexibility**: MaaS provides models for a large number of applications and allows businesses to integrate a large number of potential models, providing wide flexibility (e.g., NLP, computer vision, time series, and so many other applications). In addition, developers can test many pre-trained models quickly without changing setups (e.g., Hugging Face offers thousands of models that can be used with just a few pipelines). Similarly, MaaS providers often offer many tools to simplify the AI life cycle (data labeling, data format integration, monitoring tools, and so on) from training to deployment.

MaaS is a new paradigm, and the field of generative AI is also in active development, so there are challenges and possible drawbacks that need to be addressed:

- **Security and privacy**: Often, a large amount of data is transferred, especially for model training, which can be intercepted. In addition, models trained on sensitive data can end up outputting sensitive data. These models could also be trained on copyrighted data, and the legislation on training with such data is not entirely clear. So, organizations that adhere to particularly regulated industries may not adopt MaaS. Data is the basis of these models, but the models could be trained on, or become biased due to, low-quality data. Often, there is no information on what data these models were trained on. In these cases, both the platform and the businesses using these models may be subject to fines or other regulations.
- **Vendor lock-in**: MaaS providers use proprietary tools and APIs, which does not make it easy to change from one provider to another (e.g., changing providers complicates model integration or exporting models that have been fine-tuned). This difficulty can reduce flexibility and innovation and can make a business dependent on a single provider. There may be downtime or service disruption that impacts built applications. It also makes it more difficult to experiment locally.
- **Limited customization**: Not all MaaS providers allow the fine-tuning or modification of pre-trained models. Pre-trained models may not be suitable for some particular operations, or a business may need to have control over hyperparameters and infrastructure. In addition, MaaS providers may make changes or plan updates that impact the business or no longer allow some core features of their applications.
- **Interpretability of model and results**: A model is often a black box, and a user cannot access the decision-making process. Especially for GenAI models, it is difficult to understand how the model processes the input and gets the output. For sensitive applications, this could cause problems, especially when the model produces hallucinations or incorrect outputs. In addition, the lack of transparency of the platforms may affect the ability to diagnose errors or know how to correct them.
- **Performance and cost**: Latency refers to the time elapsed between a request and its corresponding response. The latency of models depends on the underlying infrastructure, which can experience strain during periods of peak usage. Shared multi-tenant environments in MaaS platforms can lead to resource bottlenecks during peak usage times. Businesses may encounter a considerable increase in latency that makes their applications unusable. MaaS allows pay as you go, but large-scale training or inference can quickly become expensive.

MaaS remains an expanding paradigm for several businesses. For example, MaaS could have a big impact in healthcare where there are large amounts of data, and many models have already been developed. The models could be available on a platform and be used when needed by practitioners or pharmaceutical companies. Obviously, in healthcare, data security and output consistency are critical (especially if these applications are used for hospitals or other health providers). MaaS is also growing in other domains, such as finance, blockchain, and Web 3.0.

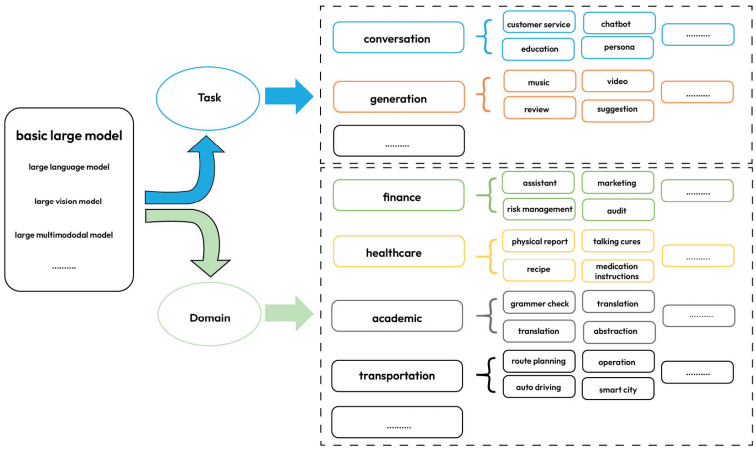


Figure 9.42 – The applications of various industries within MaaS (https://arxiv.org/pdf/2311.05804)

Data as a Service (DaaS)

DaaS is a business model where data is delivered on demand to users regardless of their geographical location or organizational boundaries. In DaaS, data is stored in the cloud and a client can access it (with or without additional tools) by paying a subscription to a provider. DaaS, therefore, is built around the concept that data is an asset and can be provided to users on demand. This access can then be conducted through a platform, the use of APIs, or additional means. In addition, the provider can provide either raw data or data that has been normalized to be machine-readable or machine-ready.

AI models are notoriously data hungry, and retrieving quality data may not be easy. So, there are players who focus on collecting hard-to-access data and then selling it to other players. For example, patient data can be difficult to collect, and a company may collect and process the data and then sell it to pharmaceutical companies. Alternatively, DaaS allows companies to create a new business model, using data collected during their normal operations as an asset they can sell. For example, a telecom company that has collected data from its users can sell the anonymized data to retailers. This data is sold through a secure portal and can be charged for on a per-access basis or through a subscription. Subscription is usually the most popular method and can be divided into three subcategories: time model, quantity-based pricing model, and pay-per-call or data type base model.

A DaaS provider may just sell the raw data it has collected, but more often it also processes it and makes it analyzable by models. Some DaaS providers aggregate different sources, process them, and thus simplify the analysis process for a client. In fact, the purpose of this data is to improve business processes and decision-making for customers, or to allow customers to train their AI models.

There can also be bidirectionality, in which the provider collects the data and harmonizes it to integrate it with its own, before making it accessible to the client again. In this way, by relating it to other data, the client can extract additional value from its own data.

DaaS has some advantages:

- **Cost efficiency**: DaaS reduces customers' need to build and maintain data infrastructure and teams. It also reduces the cost of data access because of its flexibility. Customers do not need to store data; they can directly access the data stream when they need it.
- **Ease of access**: Providing data on demand allows real-time access and saves time and expertise to obtain data information. Users do not need to know the data and the structure behind it, but they can easily learn how to use it. Also, as long as there is an internet connection, the client can always access the data.
- **Scalability**: It easily scales to accommodate increasing data needs without requiring additional infrastructure investment. Customers can easily choose the data workload they need or can handle.
- **Centralized data management**: DaaS enables consistent and centralized data storage, reducing both inconsistencies and redundancies in data. This enables simplified data governance and compliance with regulations.
- **Focus on core activities**: DaaS saves resources and time, allowing businesses to focus on extracting value from data rather than managing it. In addition, it enables better collaboration among the various team members and collaborators, which can then access the same data (in the same format).
- **Integration with other services**: DaaS makes it easy to integrate data with other services in the business, especially when it comes to analytics platforms, visualization tools, and other cloud services. Likewise, it facilitates the regular updating of datasets and allows users to have access to the most accurate and current data.
- **Data quality**: As data is centralized, data quality tends to improve. Once this data is tested, if there are no updates, there is no need for further testing.

The disadvantages of DaaS are similar to the other models associated with cloud computing:

- **Data security and privacy risks**: Obviously, the location of data on the cloud can mean that sensitive and proprietary data can be accessed by third parties or be at risk of breach. Providers must comply with regulations, which are increasingly stringent today. The costs of securing infrastructure are growing, and data piracy attacks are on the rise. In addition, although data is sold anonymized, in some cases, it is possible to reconstruct the information.
- **Dependency on providers**: DaaS creates a reliance on external providers for critical data. Service outages or disruptions on the provider's end impact the client and all services that are related to accessing this data. The client normally has access to the data stream but does not download the data, so it can be cut off from data that is necessary to its business.
- **Limited customization**: DaaS may not provide data in the format needed or have the right granularity. Providers have an interest in providing data in a format that is useful to as many clients as possible,

but specific clients may have different requirements. An inadequate format makes it more complicated to integrate into existing systems or their own workflows, requiring costs to be incurred in order to adapt either the systems or the data.

- **Quality assurance**: In DaaS, quality in terms of accuracy of data is key, and poor-quality data can lead to flawed decision-making or errors in related services. The quality, accuracy, and reliability of the data depend on the provider. Therefore, the provider must ensure that the data is relevant, updated, and of good quality.
- **Latency and performance issues**: Accessing data over the internet can lead to introducing latency (especially when the connection is not good or the datasets are very large). In addition, this latency can reduce performance if the data stream is embedded in additional services.

Results as a Service (RaaS)

RaaS, or OaaS, is a new paradigm that has developed in recent years. RaaS is a business model where a service provider delivers specific results or outcomes instead of providing tools, platforms, or raw data. This model has attracted attention in fields such as data analytics, AI, and automation. In RaaS, AI (including LLMs and agents) is used by the provider to provide personalized insights for customers. While the provider conducts the entire analysis, the client can focus on business insights without the need for specialized technology staff. In general, instead of paying a lump sum for a service, the client pays through a subscription to receive analytics at constant intervals.

Since customers increasingly demand value from models (businesses are more interested in the value obtained from models than from an additional tool), RaaS focuses on providing an outcome rather than a model (or data). In addition, customers are looking for ways to reduce the costs of adopting a technology but preserving its value, and RaaS thus seeks to reduce the initial cost to a business. The provider focuses on identifying the technology or what tool is needed to achieve the outcome, while the customer explains what their needs and requirements are.

The purpose of RaaS is to build customer loyalty, and so a provider has every interest in automating the analysis process. Therefore, AI agents can be envisioned as a new core component of this business model. By itself, an LLM is capable of almost instantaneously producing a possible report and thus generating insights for a customer. These reports can be personalized using LLMs and provide insights tailored to the clients. The addition of tools and databases allows for both adding a quantitative component and extending the capabilities of an LLM. Agents then allow tasks to be completed automatically and routinely. In fact, agents can analyze large amounts of data and can be complemented with additional models. The reports (or even presentations) generated can be used to make informed decisions.

RaaS thus has several advantages:

- **Outcome-focused approach**: The business pays only for results (and thus for the value that is delivered) and not for tools, infrastructure, and expertise. This reduces risk for a business, since it has no responsibility for either using software or conducting analysis.

- **Cost efficiency**: For the customer, there is no need to spend money to build infrastructure and expertise. Instead, the service provider can automate the process and reduce costs (it can be rather expensive for a small business). Also, the client can adopt a subscription plan at an agreed price (with the added benefit that outcome-based pricing models align costs directly with results achieved), and the provider instead gets a stable monthly income.
- **Focus on core competencies**: Since a company does not have to invest resources in building and maintaining systems or managing processes, RaaS provides a large time advantage. This also allows the business to implement new capabilities, demanding execution only from the provider. The customer can then focus on its core competencies and incorporate the results directly into its pipeline.
- **Scalability, accuracy, and flexibility**: The system is scalable and flexible, as the provider can reuse the technology for different clients. Providers are incentivized to deliver high-quality outcomes since their payment or reputation depends on the success of the service.

RaaS can also have some disadvantages:

- **Loss of control**: Clients have limited control over how these outcomes are achieved. They can't track the process or diagnose potential problems that arise during the process. In addition, there could be potential concerns over compliance, quality, or ethical practices on the part of the provider that the client might not notice. In general, RaaS does not promote transparency, and it relies on the client's trust in the provider.
- **Dependency on providers**: For customers, RaaS means heavy reliance on a service provider, which can lead to vendor lock-in, difficulty in changing providers, or high costs in changing a provider. Any failure or inefficiency on the provider's part has a direct impact on customer operations. In these cases, the customer has limited options.
- **Data security and privacy risks**: Sensitive data may need to be shared with the service provider, creating privacy and security concerns. Businesses may not be able to share this data due to regulation, risking potential breaches and hefty fines. At the same time, if sensitive data were intercepted, businesses could face serious reputational damage or fines. RaaS service providers, therefore, come with large costs to maintain system security, data storage, and connections.
- **Complexity in measuring results**: Defining clear, measurable outcomes can be challenging, especially when the goal or analysis is complex. Misaligned expectations between the client and the provider may lead to disputes about whether outcomes have been achieved. These disputes can become costly lawsuits and impact the provider's reputation.
- **Potential for higher costs**: On the one hand, RaaS reduces upfront costs, but in the long run, the service can become expensive for a business. Also, there may be added costs for further analysis, or if there is misalignment in performance and goals.
- **Limited customization**: RaaS solutions may be defined by broad application, and may not meet specific, niche requirements of a business. A service provider has every interest in automating tasks and creating solutions that are useful to the greatest number of customers. This means specific customer needs may have additional costs, not be addressed, or not be fully understood by the provider.

- **Quality assurance challenges**: The provider has an interest in reducing costs; this is done through automation and trying to achieve a solution that fits all clients. A provider may cut corners to achieve outcomes quickly, potentially compromising long-term value.

RaaS, in any case, is a growing business model, especially with the growing interest in AI and generative AI (many businesses want to integrate AI services but have neither the expertise nor the infrastructure to do so). Many companies are only interested in the outcome of the model (such as predictions for maintenance or a patient's outcome) rather than the model itself. Many businesses would be interested in tailoring the outcome to their specific needs, without needing to develop the entire process. Therefore, as competition increases, different providers are beginning to specialize in highly specific offerings for different types of industries. This drives innovation as companies strive to cover needs that are currently unmet. With more offerings, customers' needs will also evolve, allowing companies to focus on improving crucial elements of their business.

A comparison of the different paradigms

We can summarize the choice of paradigm as follows:

- **SaaS**: A provider should choose SaaS when they want to offer a steady and predictable revenue stream through subscriptions, their product is scalable to a large number of customers (thus reducing the cost of their solution), it is easy to support updates and maintenance, they have capabilities to leverage cloud infrastructure to minimize hardware costs, they can guarantee frequent software improvements, and ensure customer loyalty. A customer should choose SaaS when they need quick access to software without having to invest in hardware or maintenance, software flexibility and scalability are critical, or they prefer paying for software on a subscription basis rather than making large upfront investments. SaaS is also a good choice when customers prefer that updates, maintenance, and security are handled by an external provider or they are interested in applications that are remotely accessible (e.g., they have teams that are spread across various countries or various locations). Examples of companies using SaaS are Salesforce (a cloud-based CRM system widely used across industries), Microsoft 365 (offers productivity tools such as Word, Excel, and Teams via cloud subscription), Adobe Creative Cloud (provides access to creative tools such as Photoshop and Illustrator with continuous cloud updates), and Slack (a communication platform used by distributed teams for messaging and collaboration).
- **MaaS**: A provider should look to MaaS when they can reduce the cost of model delivery with other partners (or have a solid infrastructure), have developed high-performing AI/ML models that can serve various industries (e.g., healthcare, finance, or retail), want to monetize the developed models or expertise without sharing the algorithms, and can securely and reliably guarantee the model access. Users should consider these solutions when they require advanced AI/ML models but lack the resources to build or train them in-house, or prefer outsourcing model maintenance, retraining, and optimization rather than managing it internally. These models should also be considered when cost efficiency and flexibility are priorities, especially for start-ups and businesses experimenting with AI/ML, as well as when time to market for

AI/ML-driven applications is critical. Examples of companies using MaaS are OpenAI (provides access to GPT models through APIs for tasks such as text generation or summarization), Google Cloud AI Platform (offers models for translation, vision, speech recognition, and more), AWS SageMaker JumpStart (lets businesses quickly deploy pre-trained models for tasks such as fraud detection), and Hugging Face (through its Inference API, offers hosted access to thousands of open source models).

- **DaaS**: A provider should choose DaaS if they have access to high-value, unique datasets that can benefit multiple industries, they want to capitalize on the growing reliance on data for decision-making and analytics, they want to create an additional business opportunity for their company (e.g., selling data that has been acquired over time), they can ensure compliance with data protection regulations (e.g., GDPR or CCPA), they have the infrastructure to be able to conduct data sharing, or they provide (or intend to) added value beyond raw data, such as insights, visualizations, or integration with tools. A client should consider DaaS if they need large volumes of data but do not want to invest in storage and processing infrastructure, their business relies on external or specialized datasets (e.g., market data, weather data, geolocation data, financial data, healthcare data, and so on), they prefer flexibility in accessing different datasets and scaling, or they do not want to deal with data compliance, maintenance, and security. Examples include Snowflake (a cloud data platform that enables secure data sharing across organizations), Quandl by Nasdaq (offers financial, economic, and alternative data to analysts and institutions), Clearbit (provides B2B data for sales and marketing enrichment), and the Climate Data Store from Copernicus (offers environmental and climate datasets for scientific and commercial use).
- **RaaS**: A provider may consider RaaS if they have the appropriate infrastructure to guarantee reliable and measurable outcomes to customers, prefer to differentiate themselves by focusing on delivering value and results rather than selling products or services, can measure performance and guaranteed outcomes to the customer, and have expertise in mitigating risks and guaranteeing performance. Customers should choose RaaS when they want to achieve specific outcomes without managing the underlying processes, infrastructure, or technology; when their focus is on outcomes (e.g., performance improvement or operational efficiency) rather than tools or inputs; when they want to minimize risks by paying only for successful outcomes or results; when they lack expertise in achieving some complex and specialized outcomes; or when they want to reduce costs and spread them out over time. Examples of companies that use RaaS are Pymetrics (delivers hiring recommendations based on neuroscience and AI without exposing internal mechanisms), Afiniti (uses AI to optimize call center pairings and charges based on improved performance), Uptake (provides predictive maintenance in industrial contexts tied to uptime or efficiency gains), and ZS Associates (offers analytics-driven solutions in healthcare and pharma, charging based on KPIs and performance improvements).

The following table provides a summary of the advantages and disadvantages of each paradigm for providers and users:

Category	SaaS	MaaS	DaaS	RaaS

Advantages (provider)	- Recurring revenue model. - Scalable infrastructure - Easier software updates. - Cost-efficient development life cycle.	- Enables monetization of AI/ML models. - Scalable distribution of computational resources. - Supports various industries such as healthcare and finance. - Reduced infrastructure needs (e.g., cloud-hosted ML models). - Opportunity to expand into niche AI/ML applications.	- Data monetization opportunities. - Centralized management of data. - Predictable revenue. - Ability to leverage existing datasets. - Flexibility in serving different industries.	- Steady and predictable revenue streams. - Encourages value-based pricing for outcomes. - Differentiates offering in competitive markets. - Enables providers to focus on delivering outcomes rather than selling products. - Improved customer retention.
Advantages (User)	- Low upfront cost. - Easy access to the latest software versions. - Accessibility from anywhere. - Flexibility in subscriptions to match business needs.	- Access to advanced models without the need to build or train them. - Scalable computing power to process models efficiently. - Flexibility in using models for predictions or automation. - Cost savings by avoiding the need to	- Easy and quick access to curated, usable data. - Lower cost of ownership for data systems. - Eliminates the need for large data storage/processing infrastructure. - Flexible scaling.	- Reduced risk with outcome-based payments. - Focus on results without worrying about underlying infrastructure. - Predictable performance and value. - No need for large initial investments. - Simplifies achieving desired results with expert support.

		build in-house AI/ML in-frastructure - Enables faster time to market for AI-powered ap-plications.		
Disadvantages (provider)	- High com-petition and customer churn. - Ongoing costs for in-frastructure and updates. - Challenges with region-al regula-tions and compliance.	- High ini-tial develop-ment cost for models. - Ensuring fairness, re-liability, and compli-ance in AI/ML mod-els is challenging. - Managing perfor-mance ex-pectations of models across di-verse use cases. - Resource-intensive model up-dates and retraining.	- Privacy/security concerns with data usage. - Infrastructure for real-time data delivery. - Need for com-pliance with complex data regulations (e.g., GDPR).	- Revenue de-pends on the successful de-livery of outcomes. - High up-front costs for perfor-mance guar-antees. - Complex measure-ment and ac-countability metrics. - Risk of low-er margins if outcomes are hard to deliv-er or expec-tations are misaligned.
Disadvantages (user)	- Dependence on internet connectivi-ty. - Data secu-rity and pri-vacy risks. - Long-term costs may exceed owning soft-ware outright.	- Dependence on third-party models. - Potential for bias or errors in AI/ML models. - May incur long-term costs if fre-quently needed. - Limited ability to	- Concerns about data ownership and vendor lock-in. - Potential for high long-term costs. - Possible over-reliance on third-party data. - Security risks with sensitive data.	- Dependence on vendor for outcome success. - Lack of transparency in how the processes achieve results. - Limited flexibility to modify out-comes during contracts. - May not suit users with

	customize models for highly specific needs. - Privacy concerns in certain AI/ML applications.	highly specific, non- standardized needs. - Costs can escalate if outcomes are not well defined.

Table 9.1 – Advantages and disadvantages for providers and users

The choice of business paradigm is an important one. Each paradigm has an impact on both a user and a business. Finding the right paradigm saves resources and increases revenue. The choice of paradigm impacts the technical choices for developing a multi-agent system.

Summary

In this chapter, we have seen how the tools we looked at in previous chapters can be added to an LLM. We saw that an LLM is capable of planning and reasoning, but it produces weaker results when it comes to execution. An LLM is capable of generating text, but at the same time, the enormous amount of information learned allows it to develop skills beyond text generation. While it is a computational waste to ask an LLM to classify an image, an LLM can use a specialized model to solve the task. As we saw with HuggingGPT, a model can invoke other models to identify a pizza in an image. In that case, we saw an LLM invoke more than one model, collect their outputs, and conduct reasoning about the results (observe that the models agreed on the type of pizza in the image). The LLM can then conduct reasoning and choose which models need to run to complete the task, collect the outputs, and observe whether the task is completed.

This concept makes it possible to revolutionize various industrial applications. For example, a customer can request by email to exchange an item because the size they purchased was too small. An LLM understands the complaint, devises a plan, and executes it. The model can use tools to verify the purchase, another tool to see whether the size up is in stock, software to order the shipment, and, once the order is complete, respond to the customer that their request has been fulfilled. Agents therefore enable the automation of various tasks, as they allow an LLM to use other tools necessary for task completion. As we have seen, this approach extends to many other applications: agents in the law field, agents for research in chemistry and biology, and so on. For example, AI agents could be legal assistants to help write papers, assist professors in creating lectures, or help researchers define scientific hypotheses.

Although these seem like advanced scenarios, it must be understood that LLMs have limitations in reasoning, and at present, they can automate simple tasks but not yet complex business needs. For this, there needs to be human oversight, and developers need to be aware of what the limitations of the system are. In addition, LLMs consume resources, and these systems can be computationally expensive. Scalability is one of the main

issues for a business that wants to adopt agents. Therefore, in the last section of this chapter, we discussed the various business paradigms that open up with the arrival of LLMs. SaaS is the classic paradigm that has dominated the last three decades; it was conceived during the internet revolution but before the arrival of AI as a mass product. DaaS focuses on AI and businesses' need for quality data to make informed decisions. MaaS is dedicated to companies that want to provide ML and AI models, while RaaS focuses only on the output of these models. There are clear similarities between SaaS and these paradigms, but they take into consideration two factors: AI models require infrastructure and resources to train and use, and developing and maintaining these models requires considerable expertise. MaaS and RaaS thus allow a business to reduce the initial investment into infrastructure, training, and expertise. The choice of provider or client is different depending on their needs and resources, so we have provided a comparative table and some guidelines.

In this chapter, therefore, we have defined what an agent is in practice (or a group of agents in the case of a multi-agent platform) and discussed how these agents can be integrated into the business. In other words, we have defined an agent-based system. This system is not an isolated entity; in the next chapter, we will focus on the ecosystem around an agent and how an agent integrates into it.

Further reading

- Shen, *HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face*, 2023, **https://arxiv.org/abs/2303.17580**
- Wang, *A Survey on Large Language Model based Autonomous Agents*, 2023, **https://arxiv.org/abs/2308.11432**
- Raieli, *HuggingGPT: Give Your Chatbot an AI Army*, **https://levelup.gitconnected.com/hugginggpt-give-your-chatbot-an-ai-army-cfadf5647f98**
- Schick, *Toolformer: Language Models Can Teach Themselves to Use Tools*, 2023, **https://arxiv.org/abs/2302.04761**
- Bran, *ChemCrow: Augmenting Large Language Models with Chemistry Tools*, 2023, **https://arxiv.org/abs/2304.05376**
- Cui, *Chatlaw: A Multi-Agent Collaborative Legal Assistant with Knowledge Graph Enhanced Mixture-of-Experts Large Language Model*, 2023, **https://arxiv.org/abs/2306.16092v2**
- Hamilton, *Blind Judgement: Agent-Based Supreme Court Modelling With GPT*, 2023, **https://arxiv.org/abs/2301.05327**
- Cheng, *Exploring Large Language Model based Intelligent Agents: Definitions, Methods, and Prospects*, 2024, **https://arxiv.org/pdf/2401.03428**
- Swanson, *The Virtual Lab: AI Agents Design New SARS-CoV-2 Nanobodies with Experimental Validation*, 2024, **https://www.biorxiv.org/content/10.1101/2024.11.11.623004v1.full**
- Lu, *The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery*, 2024, **https://arxiv.org/abs/2408.06292**
- Fossi, *SwiftDossier: Tailored Automatic Dossier for Drug Discovery with LLMs and Agents*, 2024, **https://arxiv.org/abs/2409.15817**
- Si, *Can LLMs Generate Novel Research Ideas? A Large-Scale Human Study with 100+ NLP Researchers*, 2024, **https://arxiv.org/abs/2409.04109**

- Raieli, *AI Planning or Serendipity? Where Do the Best Research Ideas Come From?*, https://ai.gopubby.com/ai-planning-or-serendipity-where-do-the-best-research-ideas-come-from-f8e5e6692964
- Raieli, *A Brave New World for Scientific Discovery: Are AI Research Ideas Better?*, https://levelup.gitconnected.com/a-brave-new-world-for-scientific-discovery-are-ai-research-ideas-better-5692c5aa8182
- Schmidgall, *Agent Laboratory: Using LLM Agents as Research Assistants*, 2024, https://arxiv.org/abs/2501.04227
- Tang, *ChemAgent: Self-updating Library in Large Language Models Improves Chemical Reasoning*, 2025, https://arxiv.org/abs/2501.06590
- Raieli, *Can AI Replace Human Researchers*, https://levelup.gitconnected.com/can-ai-replace-human-researchers-50fcc43ea587
- *European Cloud Computing Platforms*, https://european-alternatives.eu/category/cloud-computing-platforms
- IBM, *What is Multi-tenant?*, https://www.ibm.com/topics/multi-tenant
- Gan, 2023, *Model-as-a-Service (MaaS): A Survey*, https://arxiv.org/pdf/2311.05804
- Abe, *A Data as a Service (DaaS) Model for GPU-based Data Analytics*, 2018, https://arxiv.org/abs/1802.01639
- Forbes, *AI Agents: The Next Frontier In Intelligent Automation*, https://www.forbes.com/councils/forbestechcouncil/2025/01/02/ai-agents-the-next-frontier-in-intelligent-automation/
- World Economic Forum, *Why Should Manufacturers Embrace AI's Next Frontier – AI agents – Now?*, https://www.weforum.org/stories/2025/01/why-manufacturers-should-embrace-next-frontier-ai-agents/
- Deng, 2023, *Mind2Web: Towards a Generalist Agent for the Web*, https://arxiv.org/abs/2306.06070

Table of contents collapsed